



BANCO DE DADOS

SECRETARIA DE
CIÊNCIA TECNOLOGIA E INOVAÇÃO





Qual o objetivo do curso?

Este curso apresenta uma visão introdutória de Banco de Dados (BD).

Ao final do curso, o aluno deverá ter assimilado os conceitos fundamentais sobre BD e Sistema de Gerência de BD (SGBD), assim como os aspectos de acesso e modelagem de um banco de dados.



O QUE VAMOS ESTUDAR?

- Banco de Dados;
- MySQL;
- Linguagem SQL;
- Sistema de Gerenciamento de Banco de Dados;
- Criação do Banco de Dados;
- Criação de tabelas;
- Inserção de valores;
- Consultas;
- Manipulação de Dados.





AFINAL DE CONTAS, O QUE SÃO BANCOS DE DADOS?

- É uma coleção organizada de informações, normalmente armazenadas eletronicamente em um sistema de computador.
- Bancos de dados são sistemas organizados para armazenar, gerenciar e recuperar dados de forma eficiente.
- Podemos exemplificar situações clássicas como uma **lista telefônica**, um **catálogo de CDs** ou um **sistema de controle de RH** de uma empresa.



BDs fazem parte do nosso dia a dia!

- Operação bancária
- Reserva de hotel
- Matrícula em um curso da Universidade
- Plataformas de streaming
- Sistemas de reservas de voos
- Aluguel de carros
- Compras online



BDs fazem parte do nosso dia a dia!

- Redes Sociais
- Serviços Bancários e Financeiros
- Saúde
- Educação
- Sistemas de Gerenciamento de Transporte
- Governo e Administração Pública



O que faz um profissional de Banco de Dados?

- Projeto e implementação de bancos de dados;
- Gerenciamento de dados;
- Análise de dados;
- Desenvolvimento de Sistemas.



Mercado de trabalho

Administrador de Banco de Dados (DBA - Database Administrator)

Desenvolvedor de Banco de Dados

Analista de Dados

Engenheiro de Dados



DADO X INFORMAÇÃO

Dado: dados são valores atribuídos a algo. Estes valores não precisam ser necessariamente números. Eles também podem ser, por exemplo, conceitos ou posições em um mapa.

Ex: nome, idade, “1234”, “abc”, etc.

Informação é um dado depois de processado e contextualizado.

“Júnior” - “Júnior é aluno do curso de Banco de Dados.”





Conceitos básicos

Dados: São as informações armazenadas no banco de dados, como nomes, números, textos, datas, entre outros.

Tabelas: Os dados em um banco de dados são organizados em tabelas. Cada tabela contém registros (linhas) e campos (colunas).

Consultas: São comandos que permitem recuperar, filtrar e analisar dados específicos de uma ou mais tabelas.



Esquema de Banco de Dados:

É uma forma de representação estruturada dos dados, segundo um determinado padrão, para que possam ser “entendidos” pelo SGBD utilizado.



Modelo de Dados

Funcionário			← Esquema
Código	Nome	Salário	
001	João	300	← Instância
002	Maria	350	
003	Paulo	325	



SGBD

Sistema Gerenciador
de Banco de Dados



SISTEMA DE GERENCIAMENTO DE BANCO DE DADOS (SGBD)

São softwares que facilitam a criação, manutenção e manipulação de bancos de dados. SGBDs são projetados para gerenciar grandes grupos de informações.

É uma coleção de programas que permitem aos usuários definirem, construírem e manipularem uma base de dados para o uso das diversas aplicações.

Exemplos incluem MySQL, PostgreSQL, Oracle Database e SQLite.



Vantagens do uso de um SGBD

- Redução do esforço humano (desenvolvimento e utilização);
- Reduz a carga de trabalho humana;
- Redução da redundância e da inconsistência de informações;
- Redução de problemas de integridade;
- Compartilhamento de dados;
- Facilitam a criação de cópias de segurança regulares dos dados.

Principais SGBDs

- MySQL
- PostgreSQL
- SQL Server
- Oracle
- MariaDB



Linguagem de programação

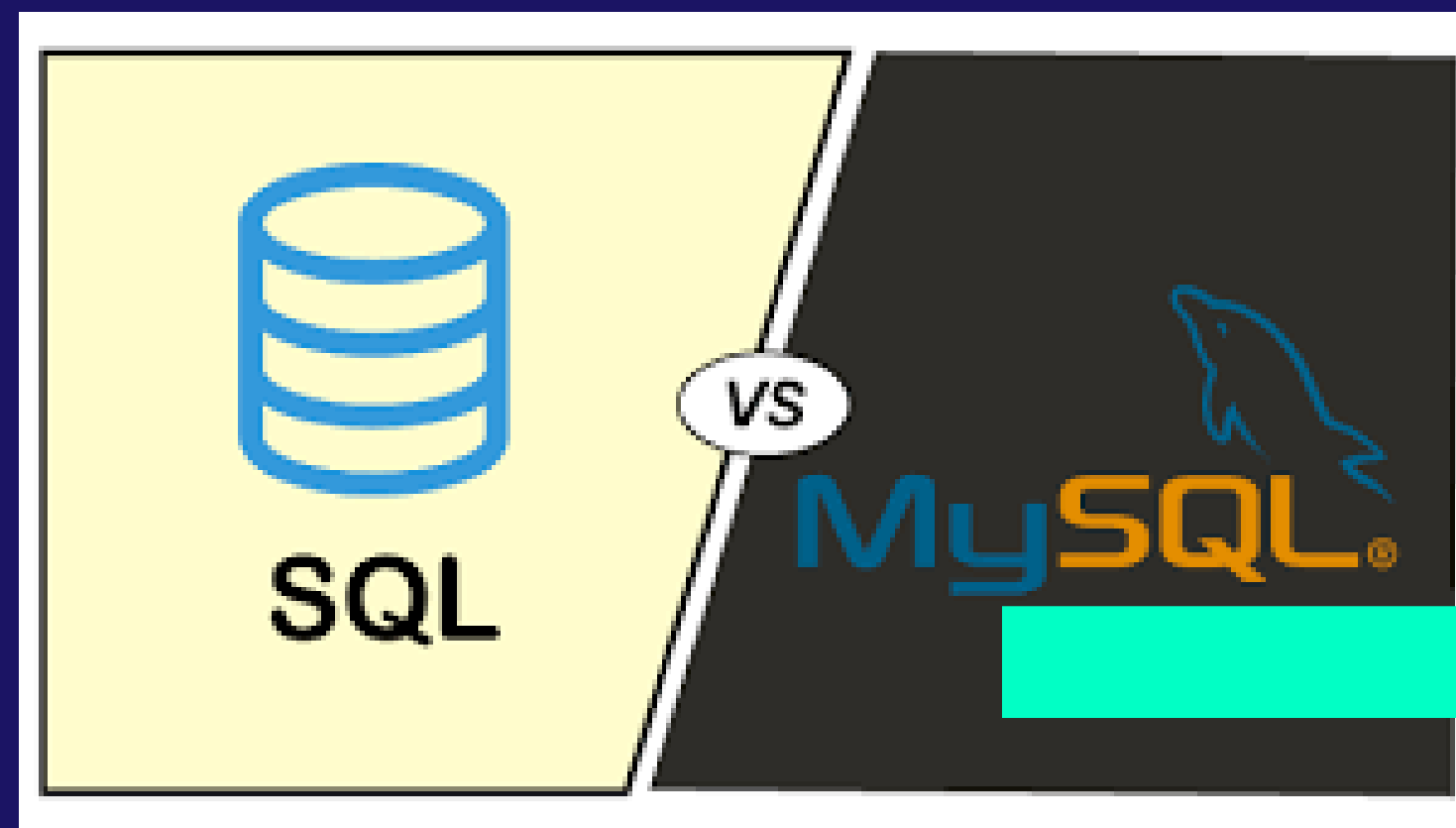
Uma linguagem de programação é uma linguagem formal utilizada para expressar instruções que podem ser executadas por um computador.

Cada linguagem de programação possui uma sintaxe específica que determina como os comandos devem ser escritos.



Exemplos de linguagens de programação incluem **Python, Java, C++, JavaScript, PHP**, entre muitas outras. A escolha da linguagem depende dos requisitos do projeto, do domínio de aplicação e das preferências do desenvolvedor.





AULA 2

Introdução a SQL



A linguagem SQL

Linguagem de consulta estruturada



- SQL foi desenvolvida pela IBM nos anos 1970.
- SQL é projetada para trabalhar com sistemas de gerenciamento de banco de dados (SGBD).
- SQL abrange diversas categorias, incluindo DDL (Linguagem de definição de dados) e DML (Linguagem de manipulação de dados).
- Sua principal função é consultar e manipular dados armazenados em tabelas.

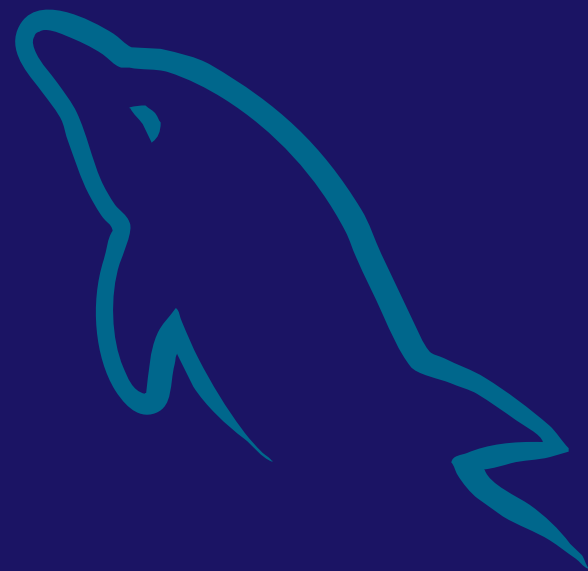


O que é o MySQL?



Por que utilizar o MySQL?

- **Desempenho**
- **Compatibilidade com Padrões SQL**
- **Multiplataforma**
- **Ferramentas de Gerenciamento**





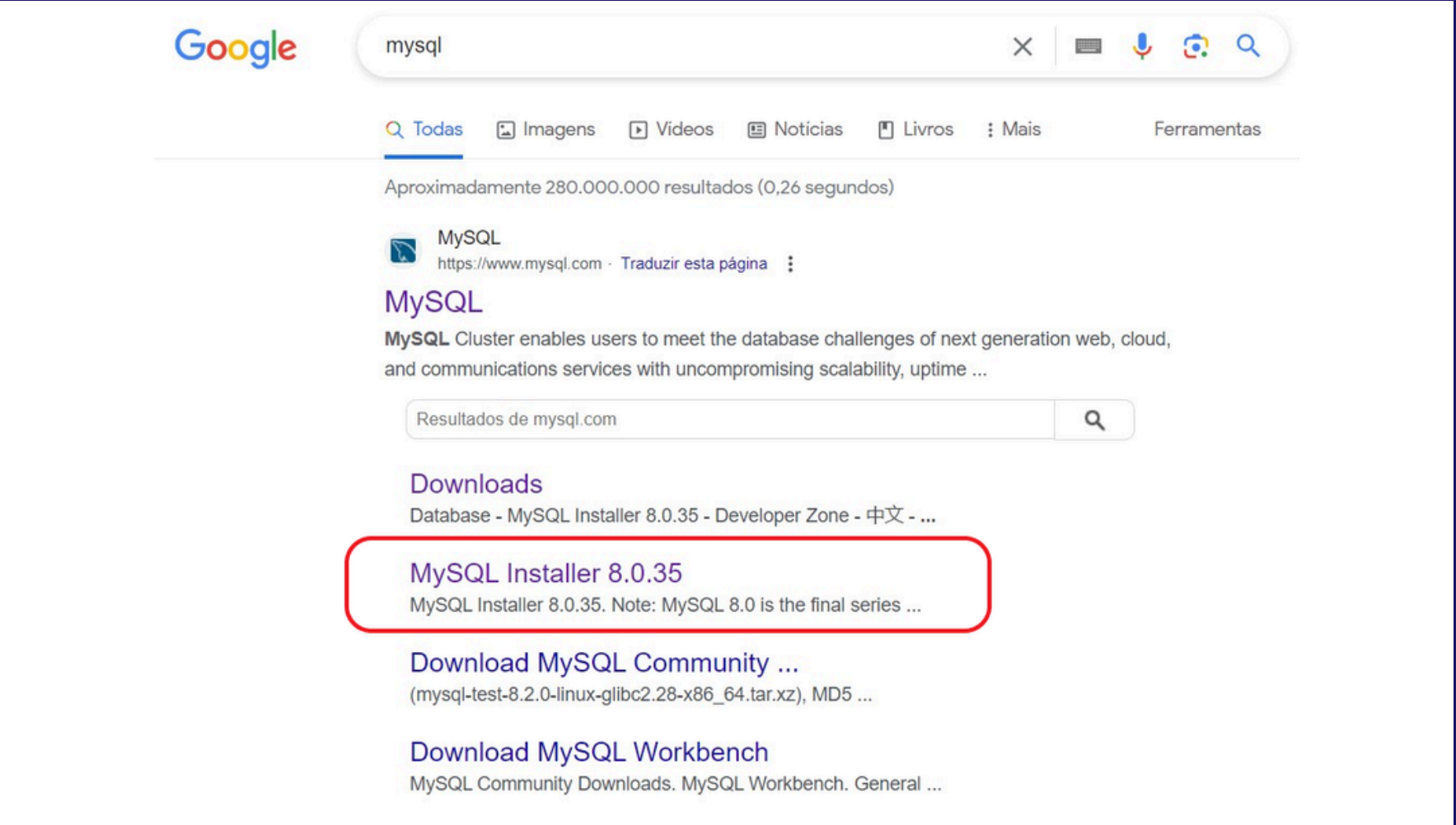
MySQL: download

O MySQL installer pode ser adquirido nesse: [LINK](#)

Após o passo a passo, você terá um servidor MySQL e suas ferramentas em funcionamento no seu computador para aprender e praticar.



Download do MySQL



Download do MySQL

MySQL Installer

General Availability (GA) Releases

Archives

MySQL Installer 8.0.44

Note: MySQL 8.0 is the final series with MySQL Installer. As of MySQL 8.1, use a MySQL product's MSI or Zip archive for installation. MySQL Server 8.1 and higher also bundle MySQL Configurator, a tool that helps configure MySQL Server.

Select Version:

8.0.44

Select Operating System:

Microsoft Windows

Windows (x86, 32-bit), MSI Installer	8.0.44	2.1M	Download
(mysql-installer-web-community-8.0.44.0.msi)			
MD5: f48ab9b8c2db55ee39ddf534d4581676 Signature			
Windows (x86, 32-bit), MSI Installer	8.0.44	558.3M	Download
(mysql-installer-community-8.0.44.0.msi)			
MD5: 338dce4ac543dfc280664c857d265e3e Signature			

We suggest that you use the [MD5 checksums](#) and [GnuPG signatures](#) to verify the integrity of the packages you download.

Download do MySQL

⬇ Downloads da comunidade MySQL

Entre agora ou registre uma conta gratuita.

Uma conta Oracle Web oferece as seguintes vantagens:

- Acesso rápido a downloads de software MySQL
- Baixe artigos técnicos e apresentações
- Postar mensagens nos fóruns de discussão do MySQL
- Reportar e rastrear bugs no sistema de bugs do MySQL

Faça login »

usando minha conta Oracle Web

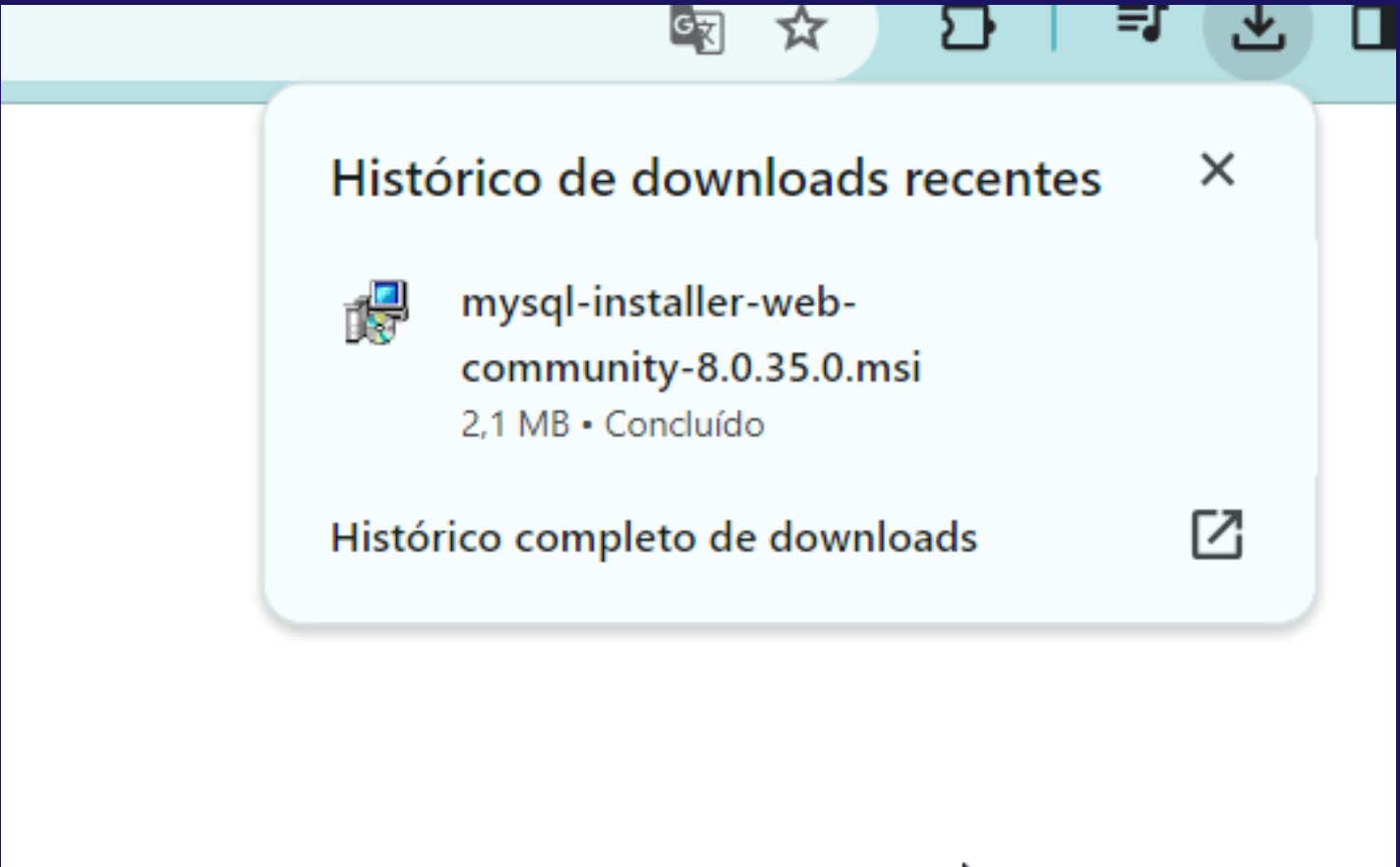
Inscreva-se »

para uma conta Oracle Web

MySQL.com está usando Oracle SSO para autenticação. Se você já possui uma conta Oracle Web, clique no link Login. Caso contrário, você pode criar uma conta gratuita clicando no link Inscrever-se e seguindo as instruções.

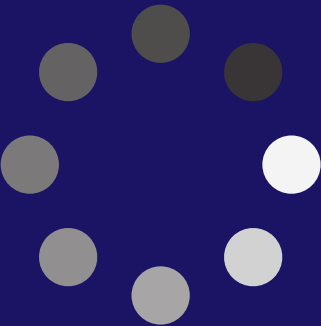
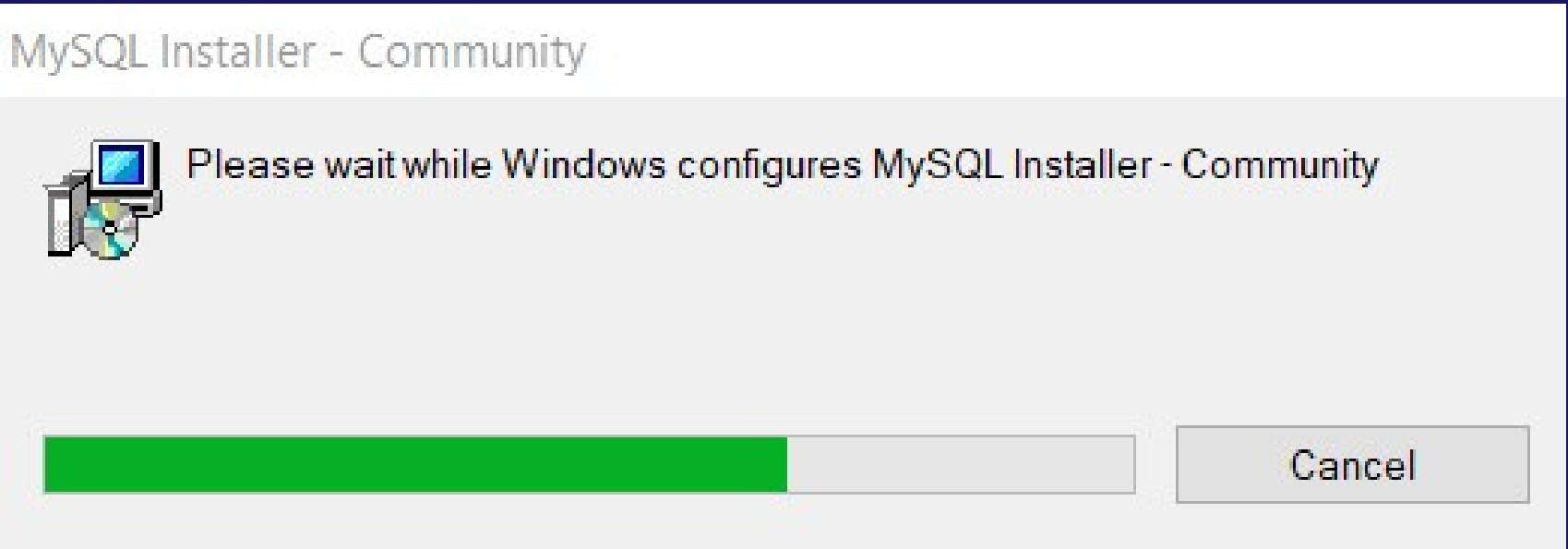
Não, obrigado, basta iniciar meu download.

Download do MySQL

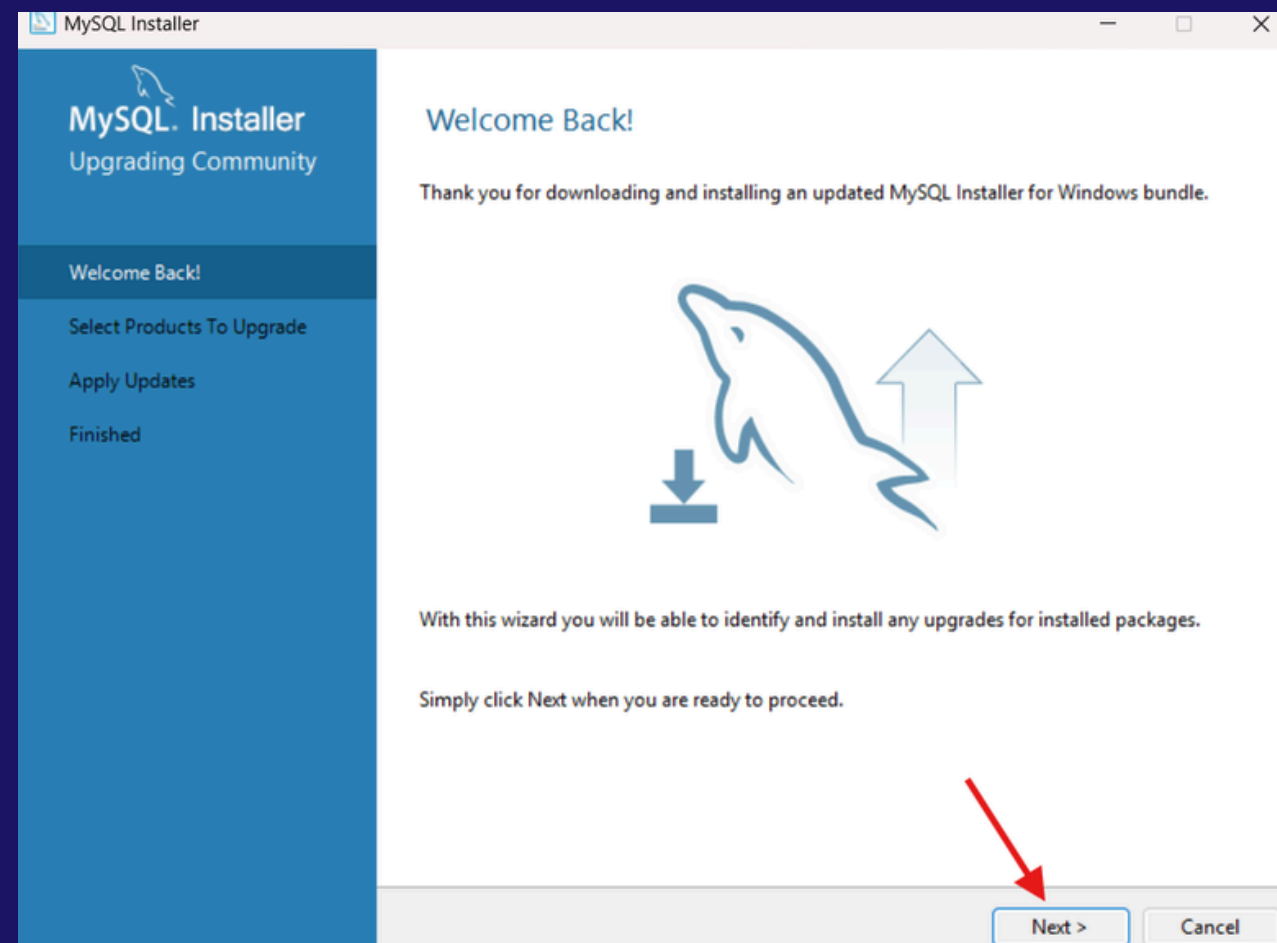


Após o download, abra o instalador.

Download do MySQL

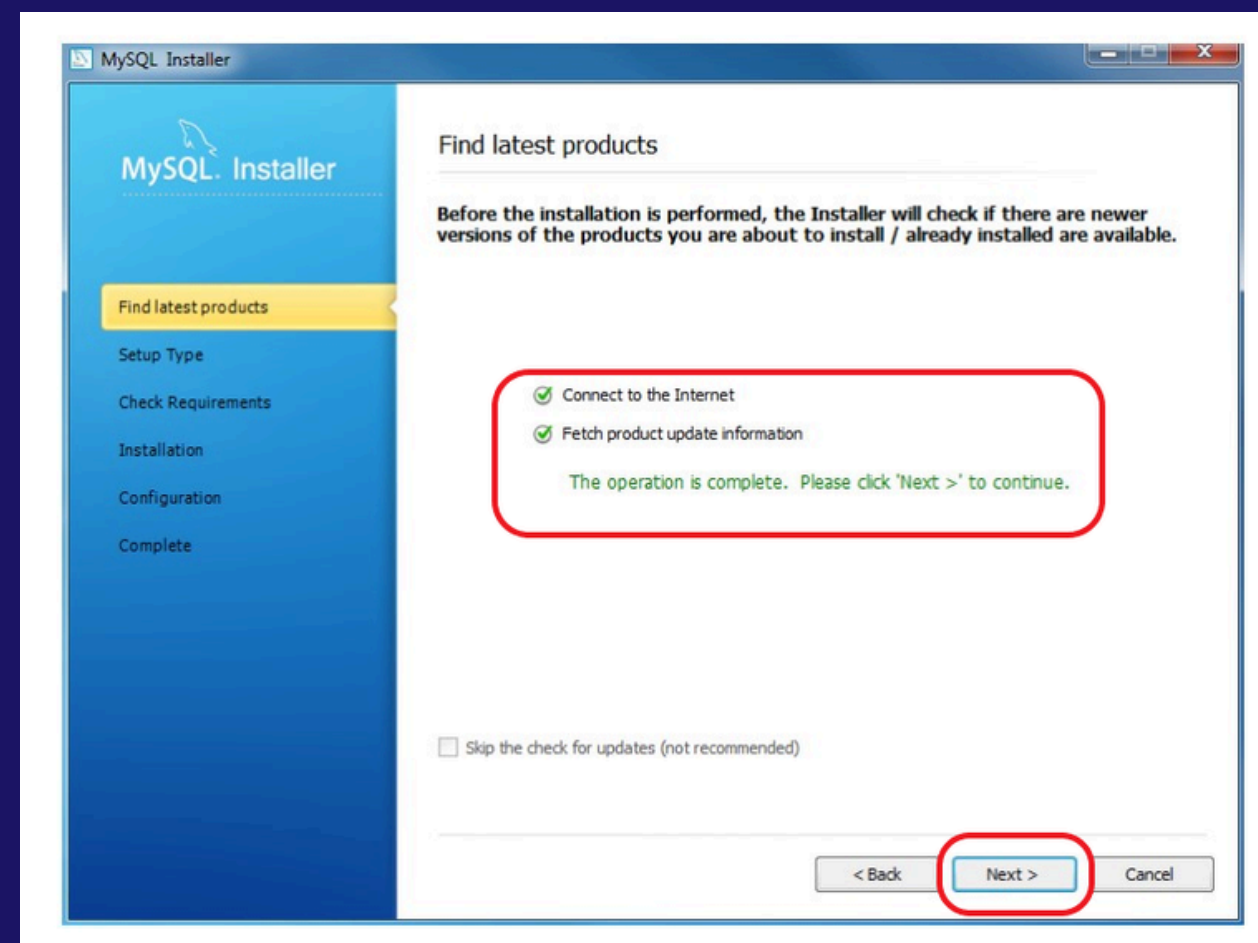


Download do MySQL



Uma tela de boas-vindas fornece várias opções. Escolha a primeira opção: Install MySQL Products.

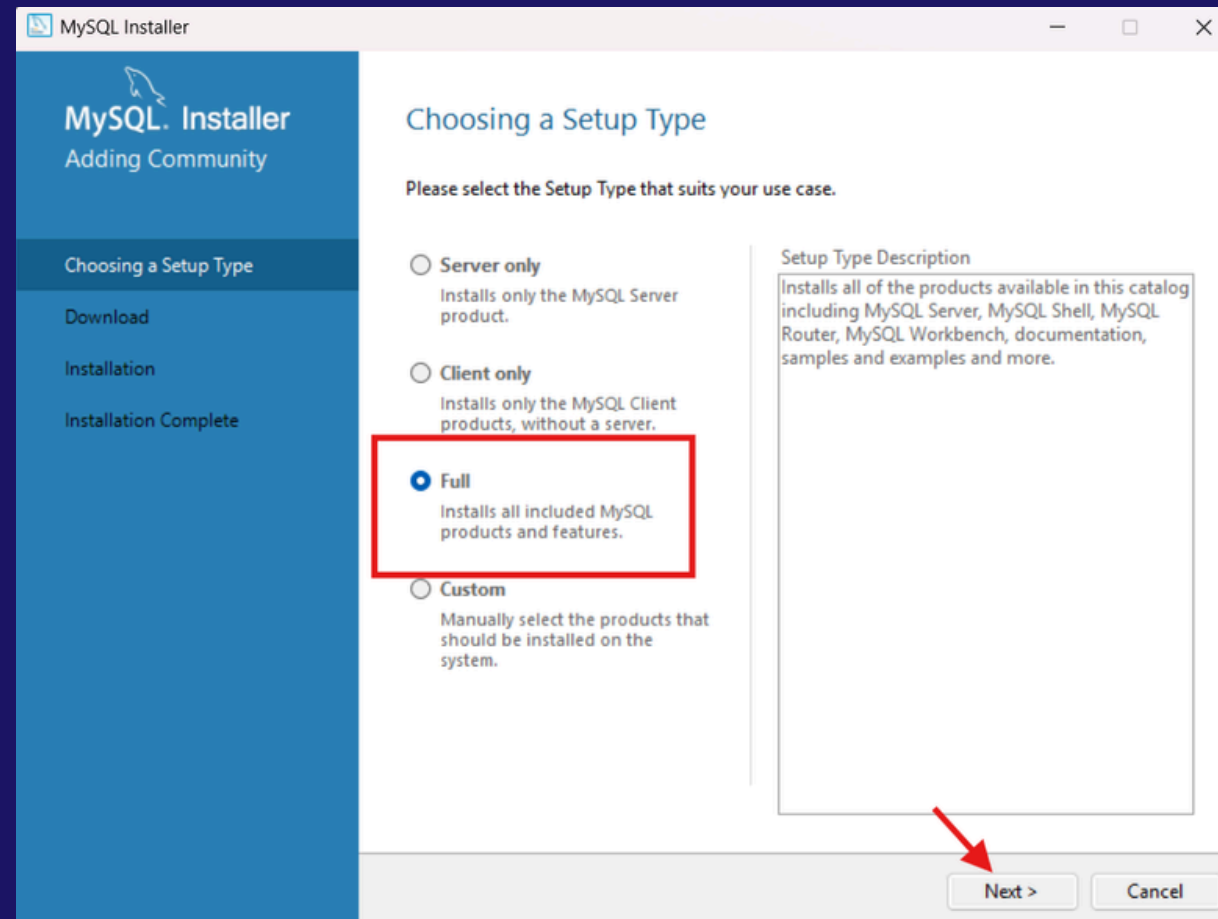
Download do MySQL



Faça o download dos mais recentes produtos MySQL: O instalador do MySQL verifica e baixa os mais recentes produtos MySQL, incluindo servidor MySQL, MySQL Workbench.

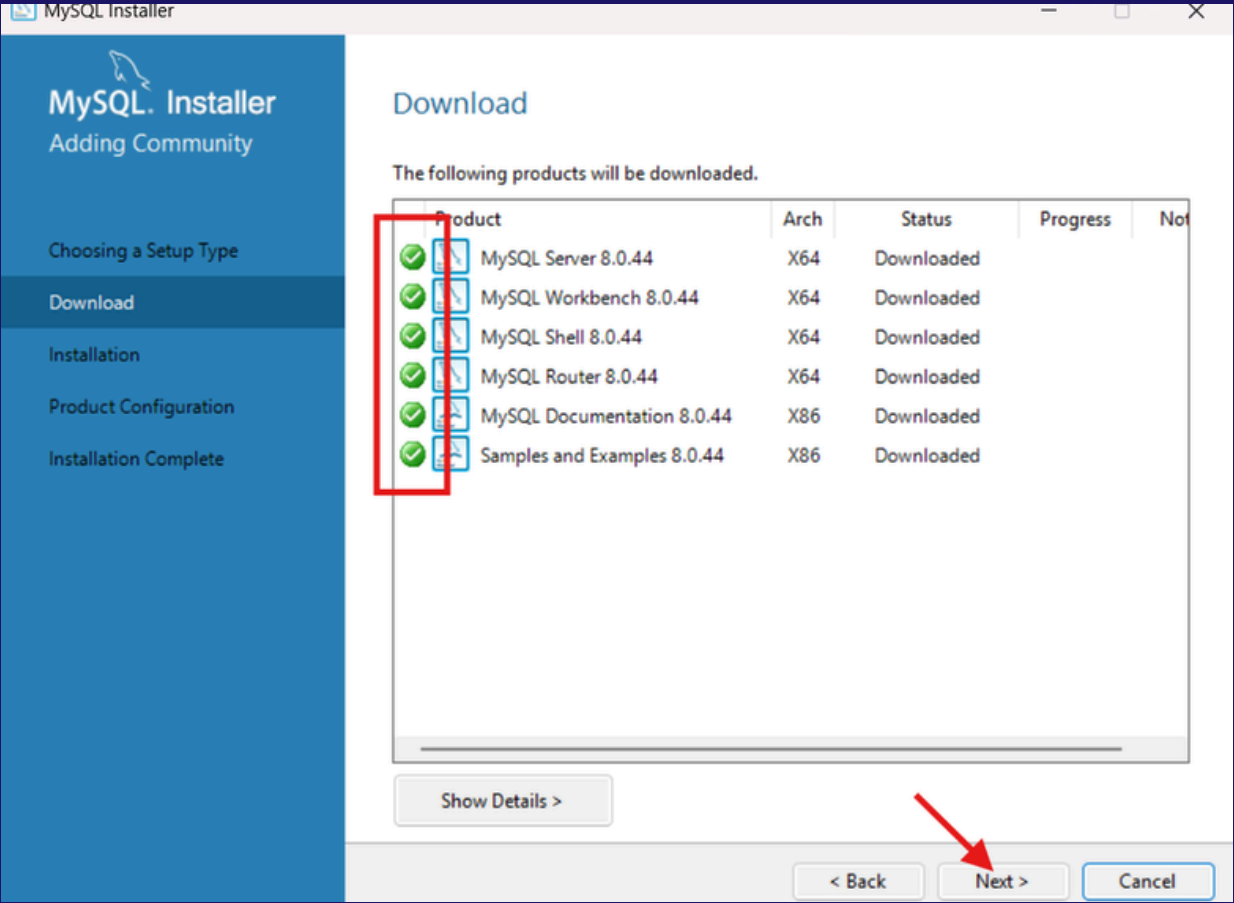
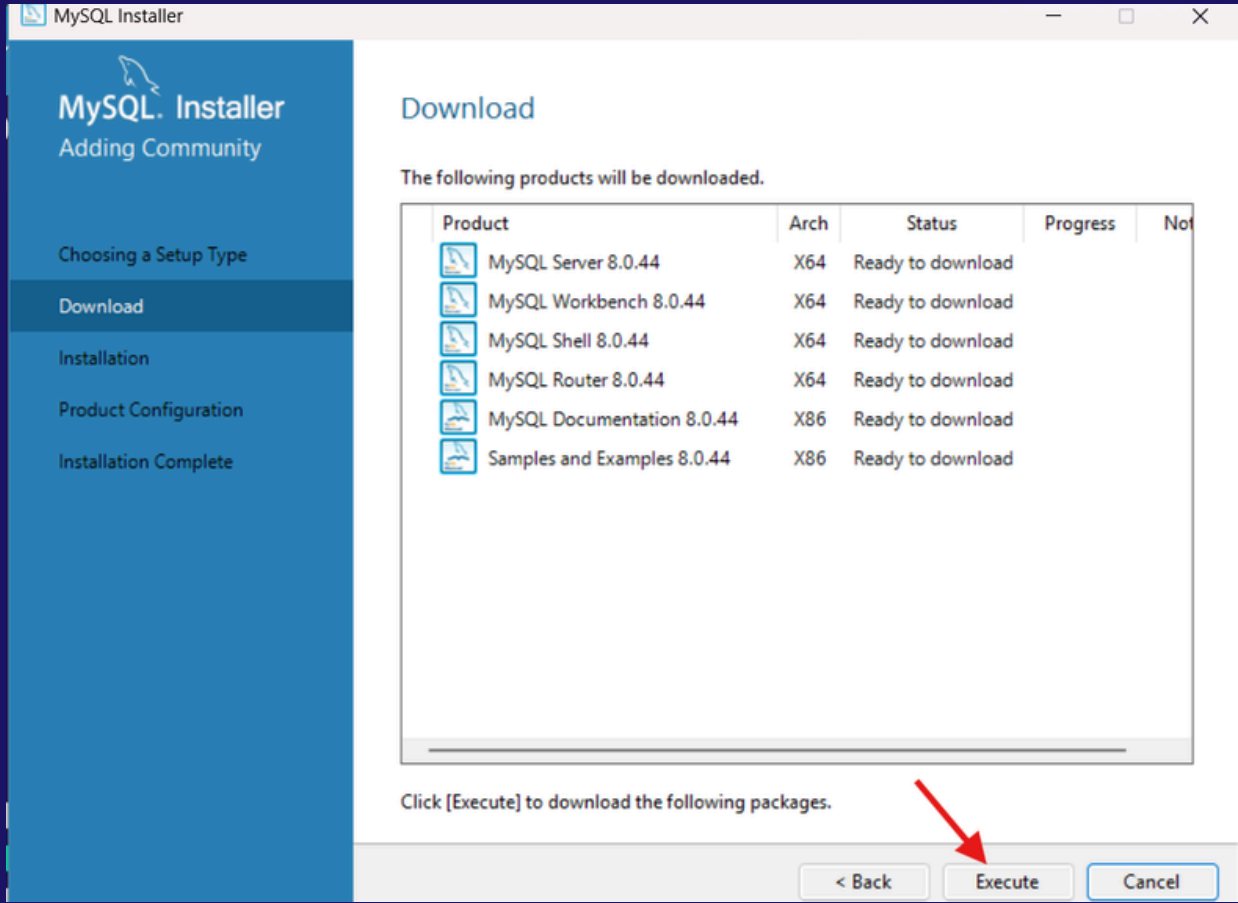
Clique no botão **NEXT** para continuar.

Download do MySQL



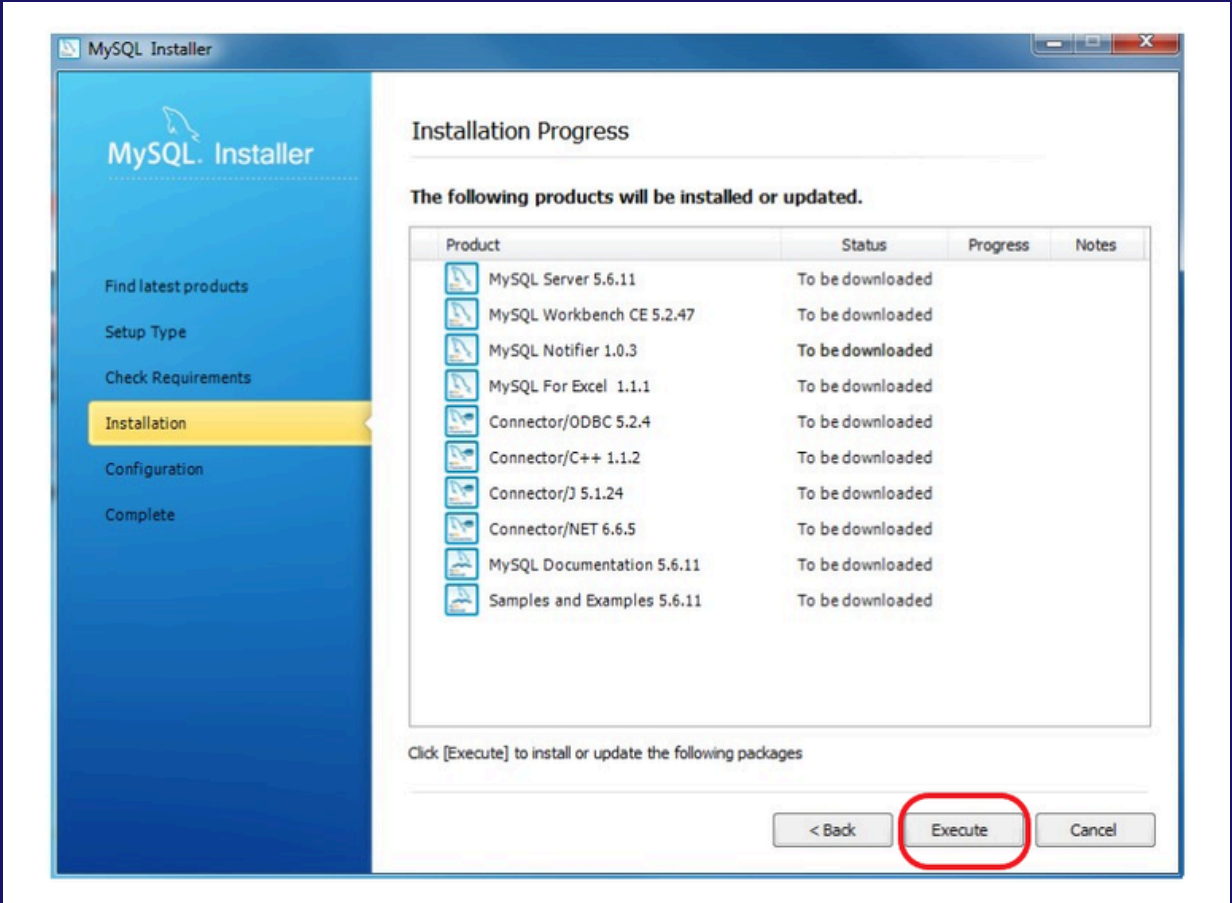
Escolha um tipo de instalação: existem vários tipos de configuração disponíveis. Escolha a opção **Full** para instalar todos os produtos e recursos do MySQL.

Download do MySQL



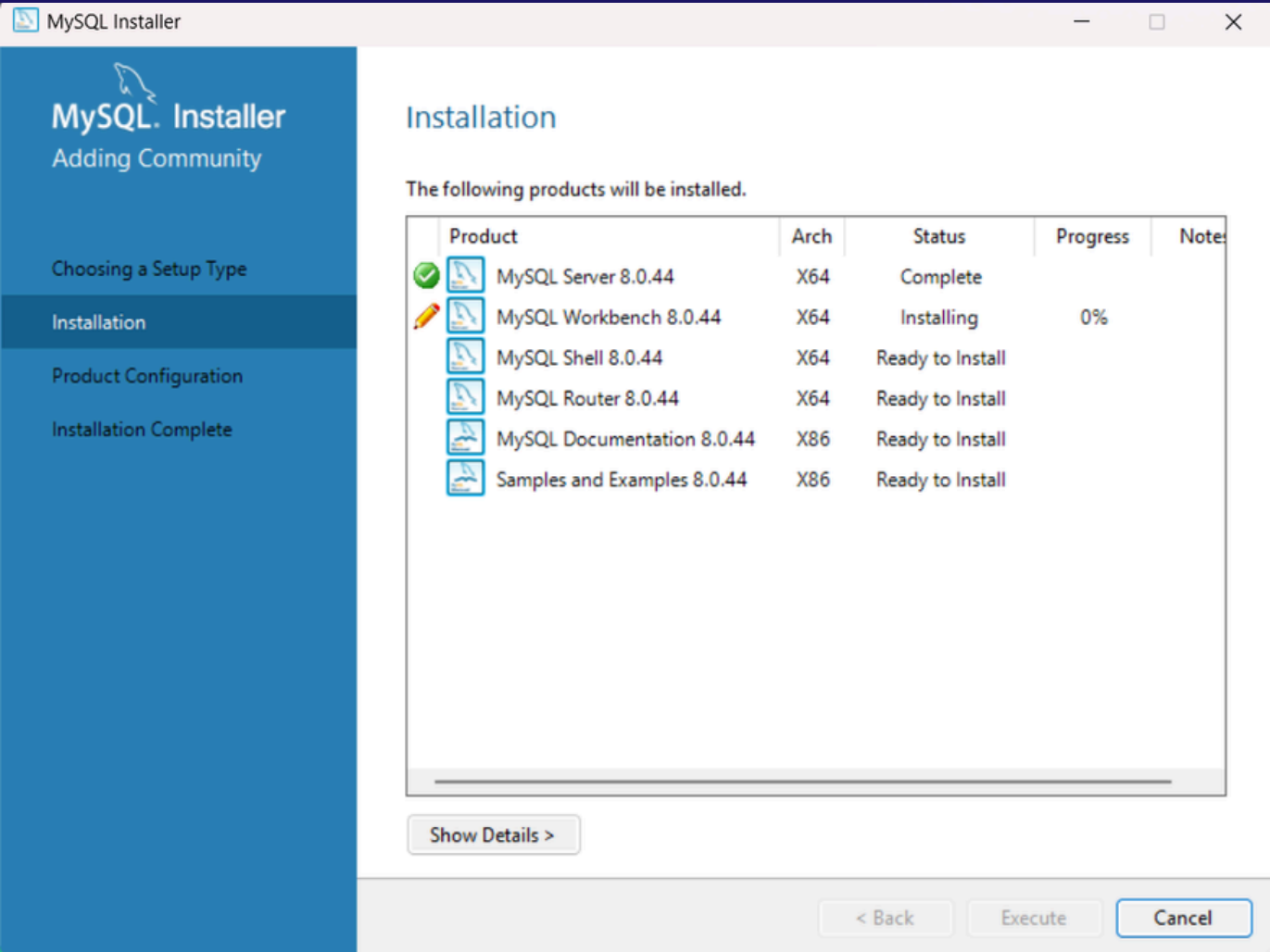
Verificando os requisitos.

Download do MySQL



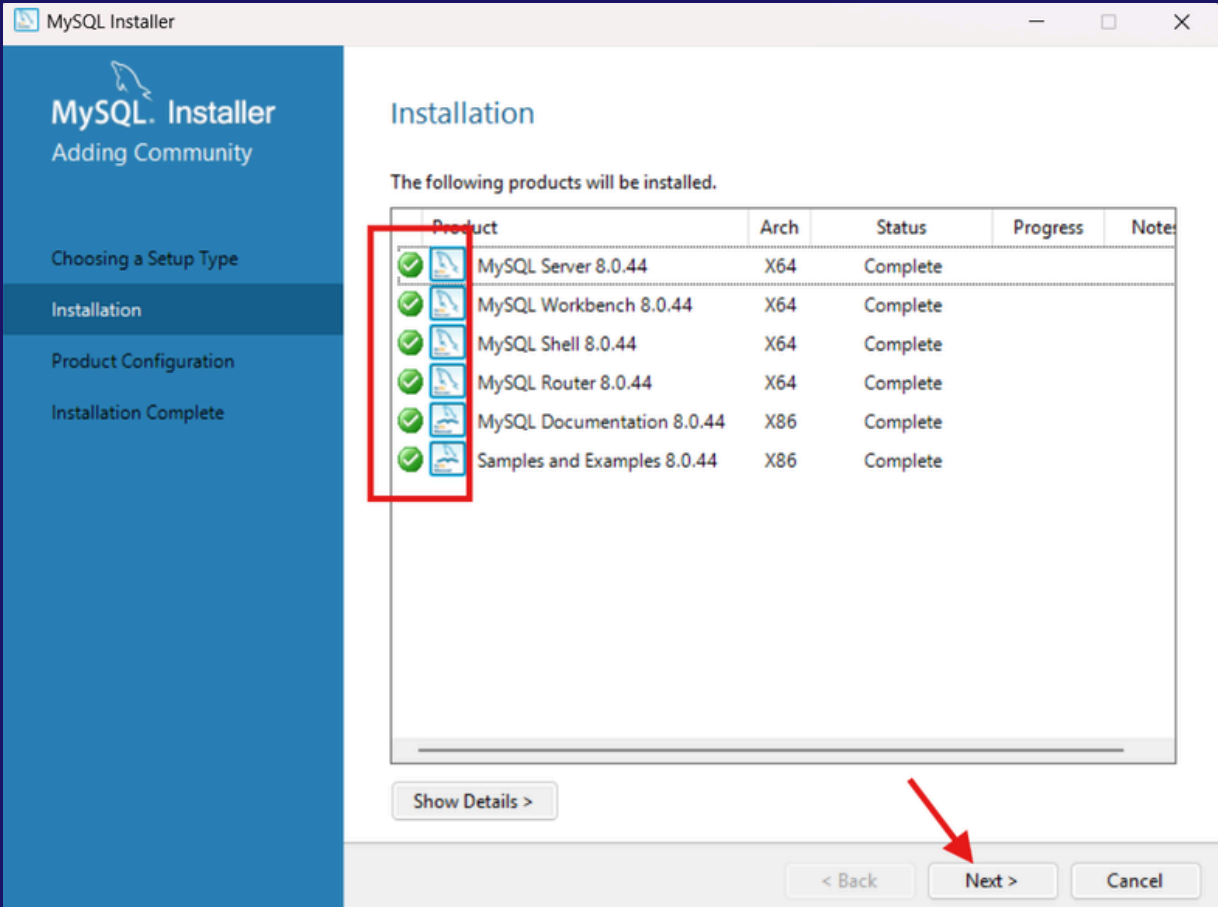
Progresso da instalação: O instalador do MySQL baixa todos os produtos selecionados. Isso poderá demorar um pouco, dependendo dos produtos selecionados e da velocidade da sua conexão à Internet.

Download do MySQL



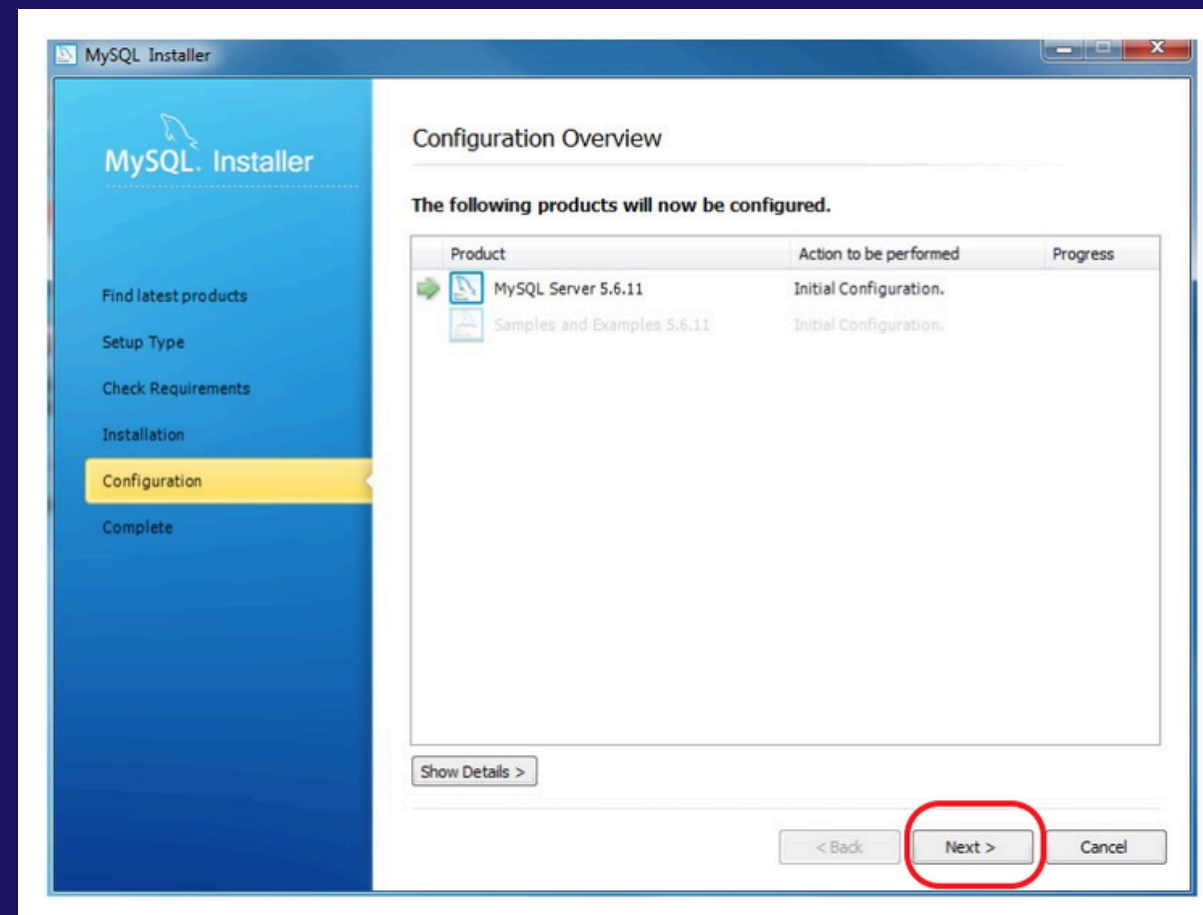
Download de produtos em andamento.

Download do MySQL



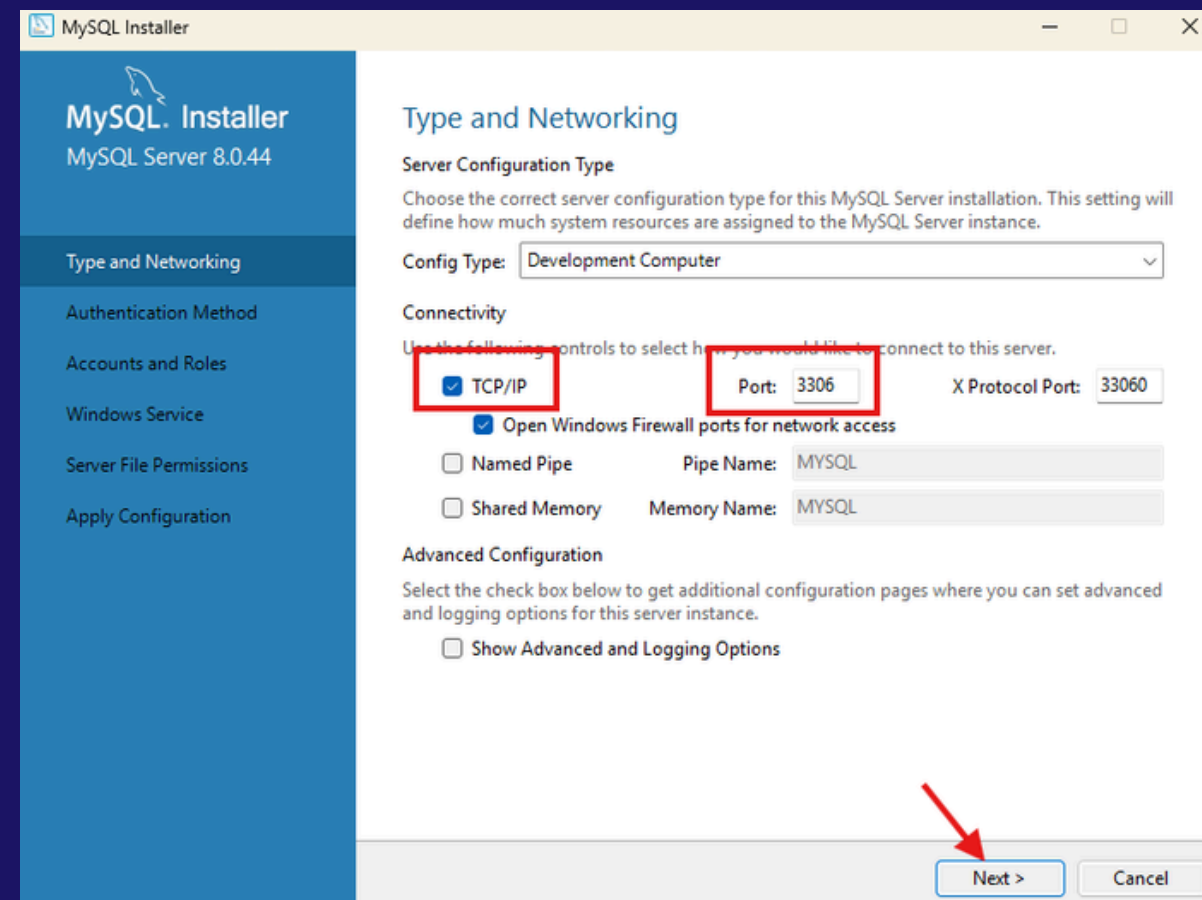
Download completo. Clique no botão **Next** para continuar.

Download do MySQL



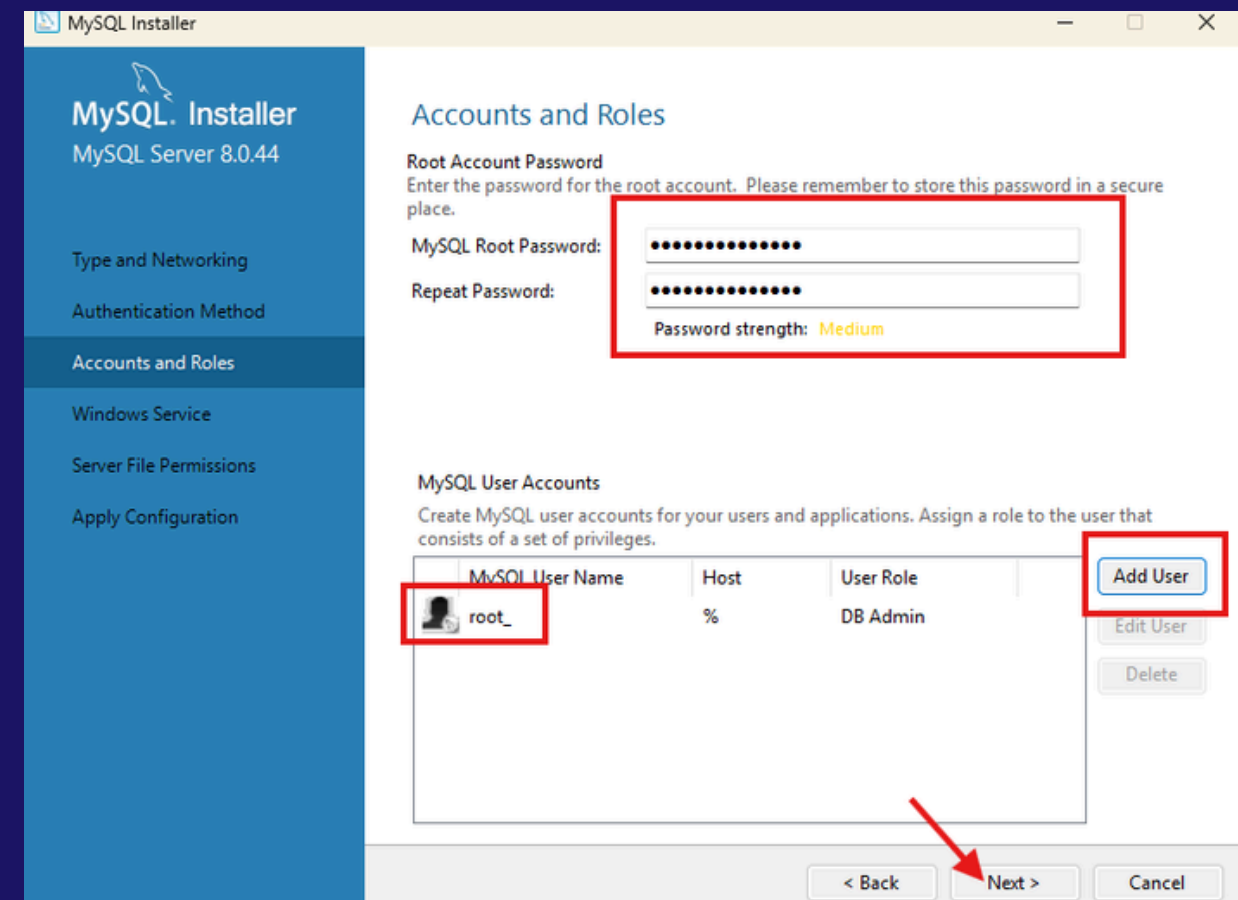
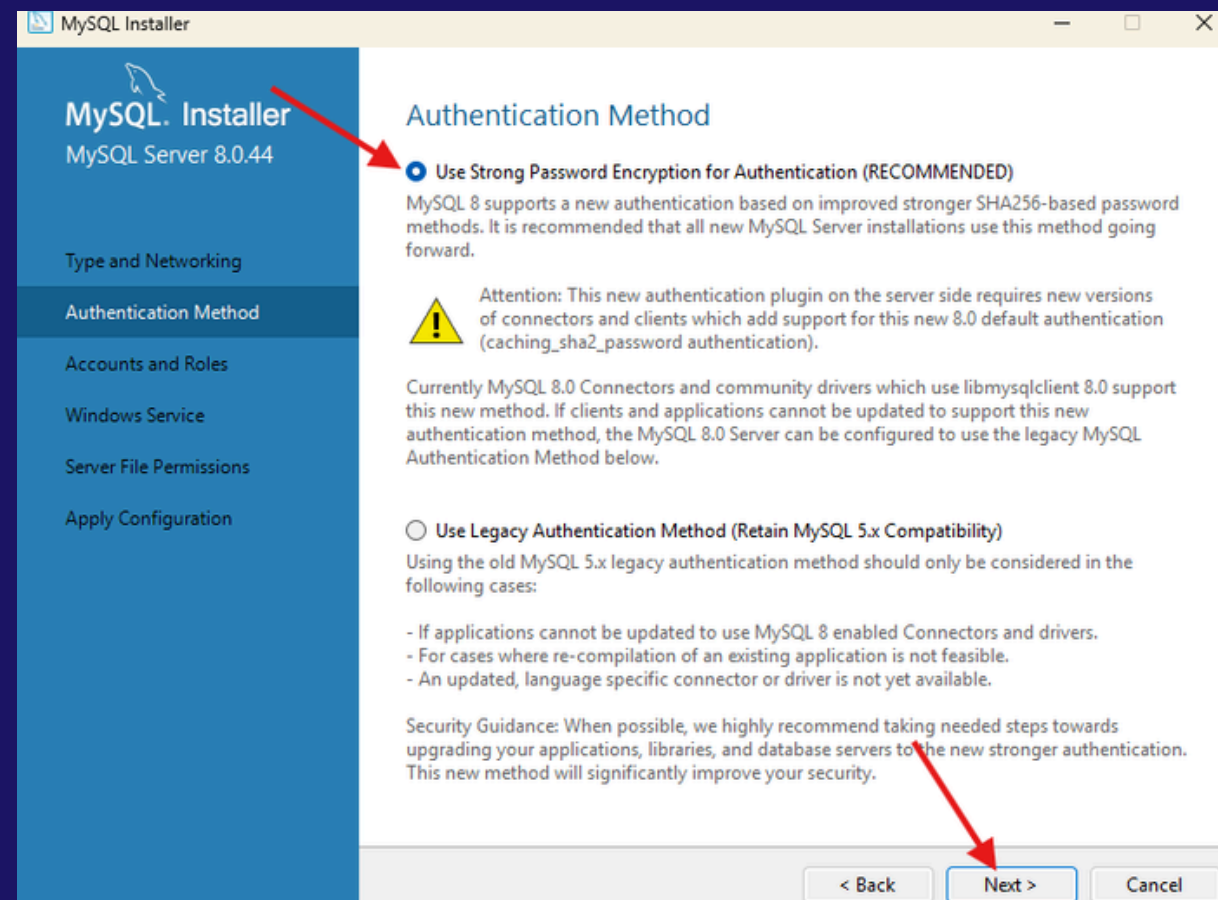
Visão geral da configuração. Clique no botão Next para configurar o MySQL Database Server.

Download do MySQL



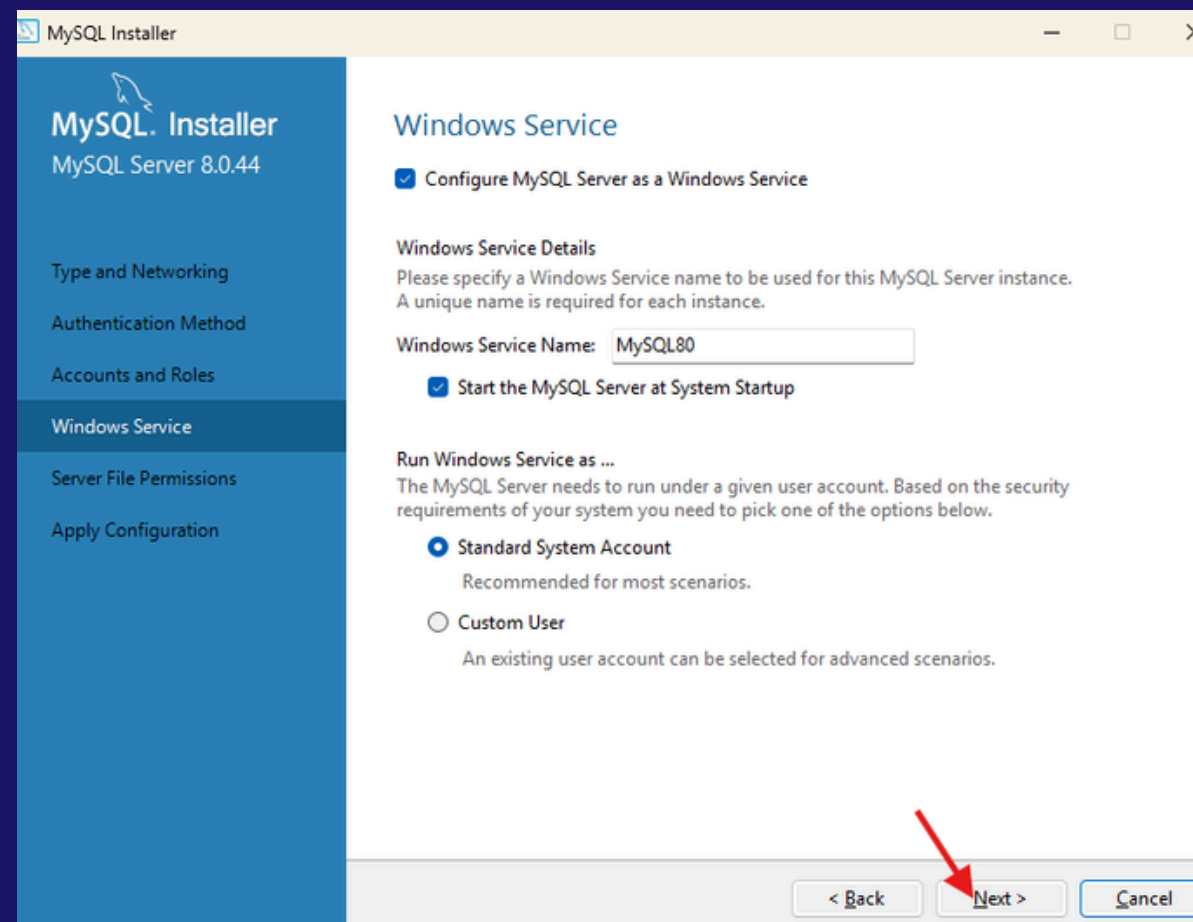
Configuração do servidor MySQL: escolha Tipo de configuração e porta MySQL (3306 por padrão) e clique no botão Next para continuar.

Download do MySQL



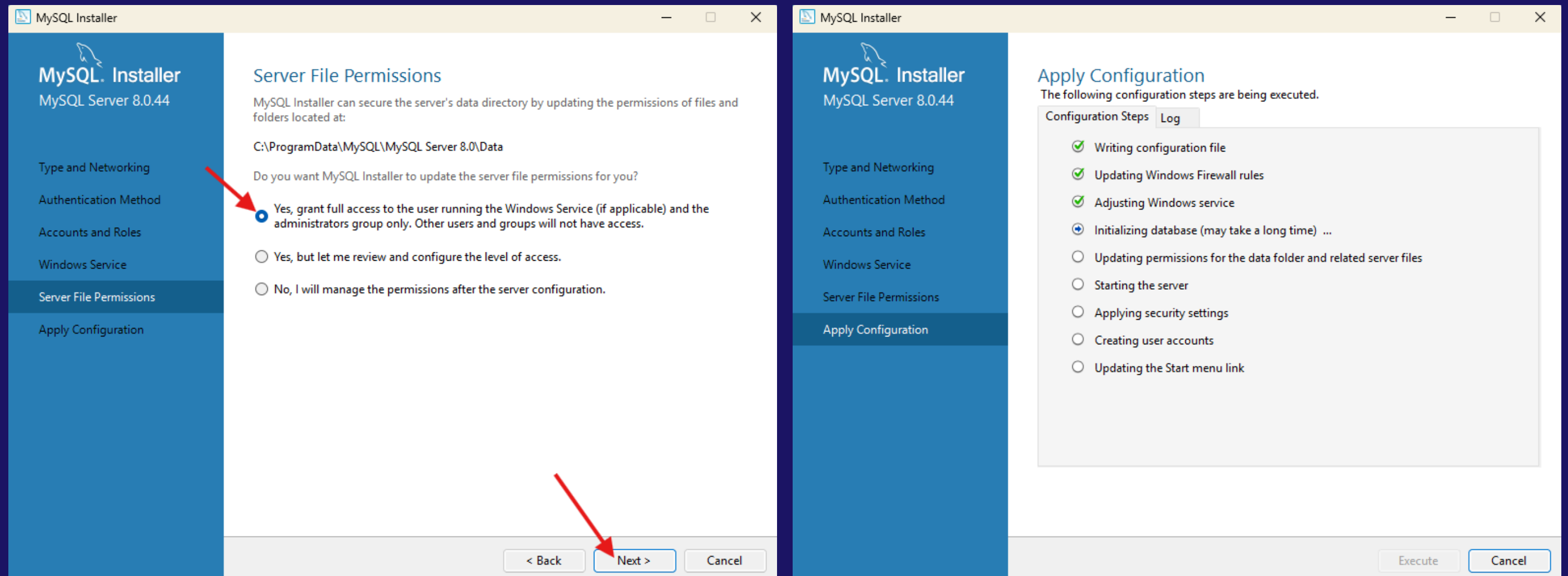
Escolha uma senha para a conta root. Anote a senha e mantenha-a em segurança se estiver instalando o servidor de banco de dados MySQL em um servidor de produção. Se você deseja adicionar mais um usuário MySQL, pode fazê-lo nesta etapa.

Download do MySQL



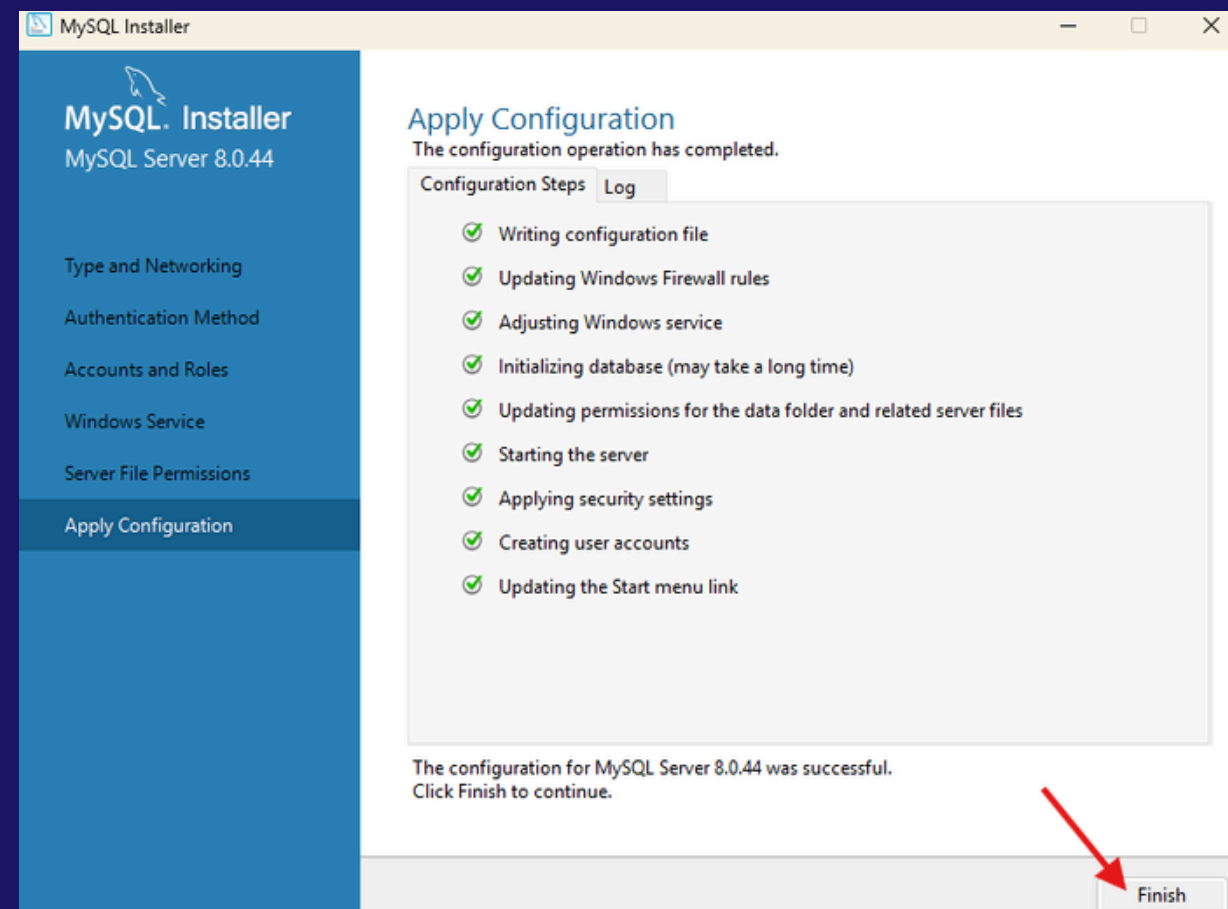
Clique em **Next** para continuar.

Download do MySQL



O Instalador do MySQL está configurando o servidor de banco de dados MySQL.

Download do MySQL



Clique no botão **Next** para continuar.

Download do MySQL

MySQL[®] Installer

MySQL Router 8.0.44

MySQL Router Configuration

MySQL Router Configuration

☐ Bootstrap MySQL Router for use with InnoDB Cluster

This wizard can bootstrap MySQL Router to direct traffic between MySQL applications and InnoDB Cluster. Applications that connect to the router will be automatically directed to an available read/write or read-only member of the cluster.

The bootstrapping process requires a connection to InnoDB Cluster. In order to register the MySQL Router for monitoring, use the current Read/Write instance of the cluster.

Hostname:

Host name or IP address is required.

Port:

3310

Management User:

root

Password:

Test Connection

MySQL Router requires specification of a base port (between 80 and 65532). The first port is used for classic read/write connections. The other ports are computed sequentially after the first port. If any port is indicated to be in use, please change the base port.

Classic MySQL protocol connections to InnoDB Cluster:

Read/Write:

6446

Read Only:

6447

X Protocol connections to InnoDB Cluster:

Read/Write:

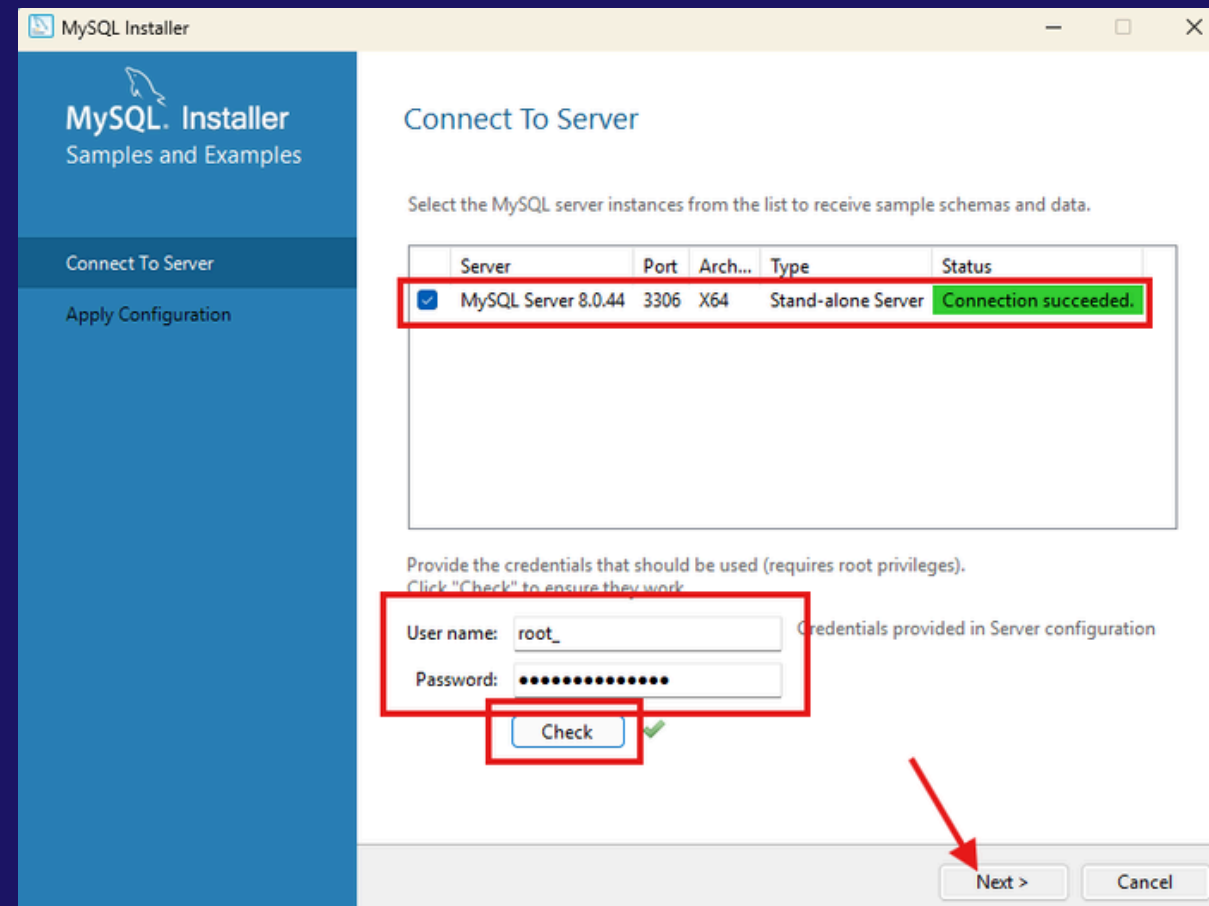
6448

Read Only:

6449

O Instalador do MySQL instala bancos de dados e modelos de amostra.

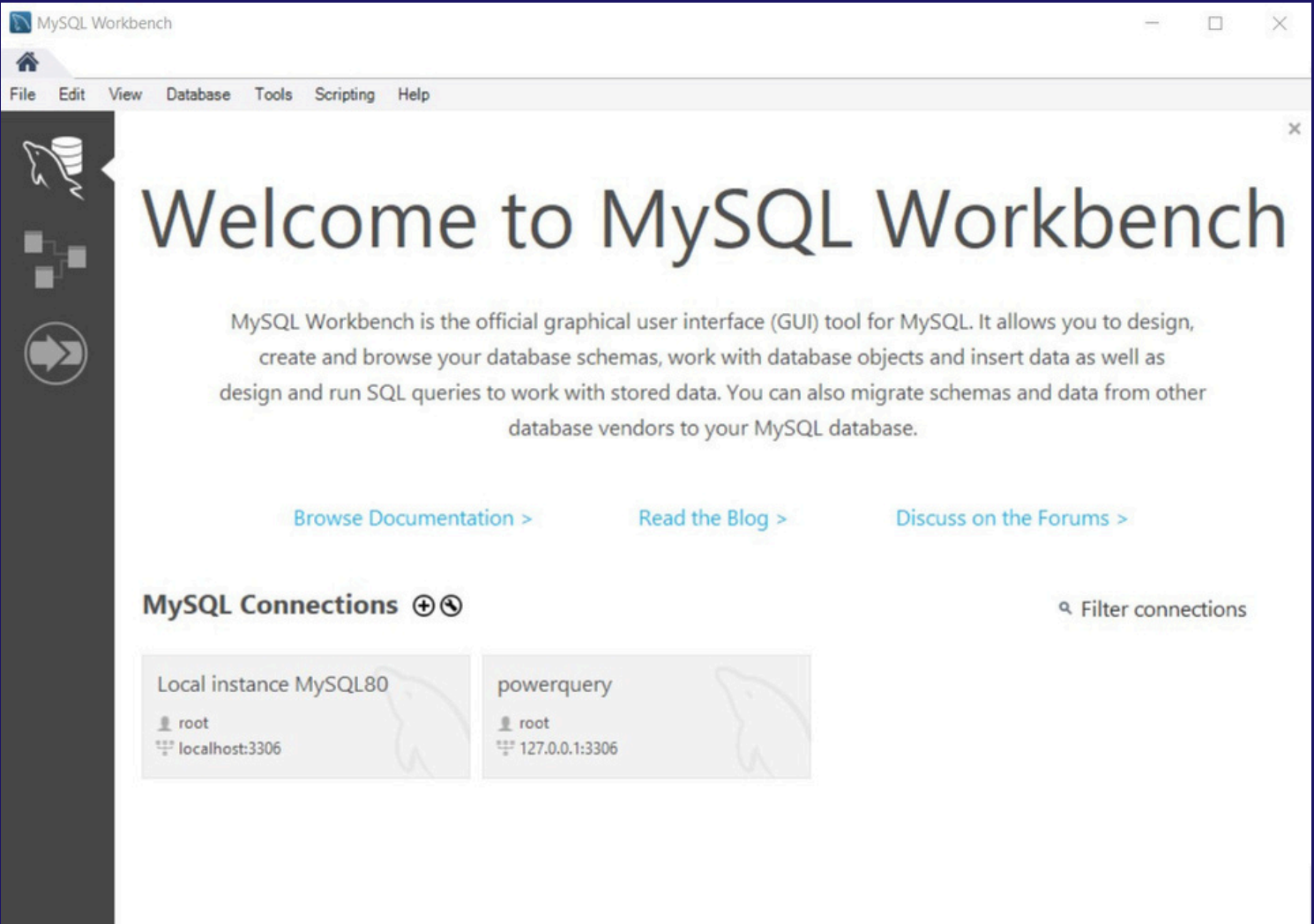
Download do MySQL



A instalação está concluída. Clique no botão Finish para fechar o assistente de instalação e iniciar o MySQL Workbench.



Download do MySQL



Introdução à Banco de Dados



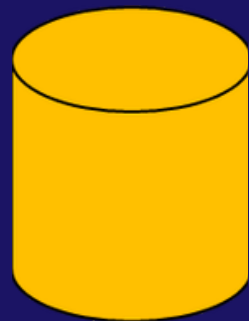
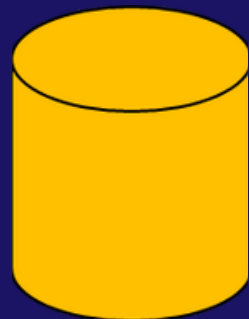
Banco de Dados



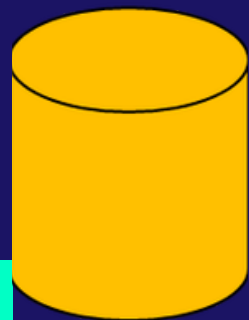
Entidades



Tabelas



...



Tipos de dados básicos



Em banco de dados relacionais, cada tabela pode conter diversas colunas, as quais armazenarão os dados. Para cada coluna, existirá um tipo de dado associado. Os tipos de dados são definidos durante a criação da tabela.

Basicamente, os tipos de dados básicos são usados para definir o tipo de informação que pode ser armazenado em uma coluna de uma tabela.



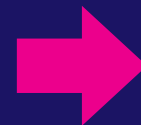
Tabelas



Campos



Registros





Tipos de Dados Primitivos

1. INT ou INTEGER
2. FLOAT ou DOUBLE
3. CHAR(n) ou CHARACTER(n)
4. VARCHAR(n) ou CHARACTER VARYING(n)



Tipos de dados básicos

5. DATE

6. TIME

7. BOOLEAN ou BOOL

8. TEXT

9. DECIMAL



A linguagem SQL



A SQL é uma linguagem declarativa, o que significa que você especifica o que deseja obter, modificar ou criar, e o SGBDR determina como realizar a tarefa.

Foi desenvolvida para interagir com sistemas de gerenciamento de banco de dados (SGBD) e realizar operações como consultas, inserções, atualizações e exclusões de dados.



DDL (Linguagem de definição de dados)

CREATE: Utilizado para criar objetos no banco de dados, como tabelas, índices e visões.

ALTER: Modifica a estrutura de objetos existentes no banco de dados.

DROP: Remove objetos do banco de dados.



PHPMyAdmin



- Ferramenta com interface simplificada e intuitiva;
- Uso diretamente em um navegador;
- executar funções básicas do MySQL, como consultar, navegar, criar, alterar e copiar tabelas, campos e índices;
- realizar manutenção dos bancos de dados e do servidor;
- carregar tabelas;
- importar e exportar dados (em CSV e SQL);
- exportar arquivos dos bancos de dados;
- administrar os servidores na interface do navegador.

Xampp

[Apache Friends](#) [Baixar](#) [Hospedagem](#) [Comunidade](#) [Sobre](#) [Pesquisar](#) [PT BR](#)

 **XAMPP** Apache + MariaDB + PHP + Perl

O que é o XAMPP?

XAMPP é o ambiente de desenvolvimento PHP mais popular

O XAMPP é completamente gratuito, de fácil de instalar a distribuição Apache, contendo MySQL, PHP e Perl. O pacote de código aberto do XAMPP foi criado para ser extremamente fácil de instalar e de usar.


XAMPP

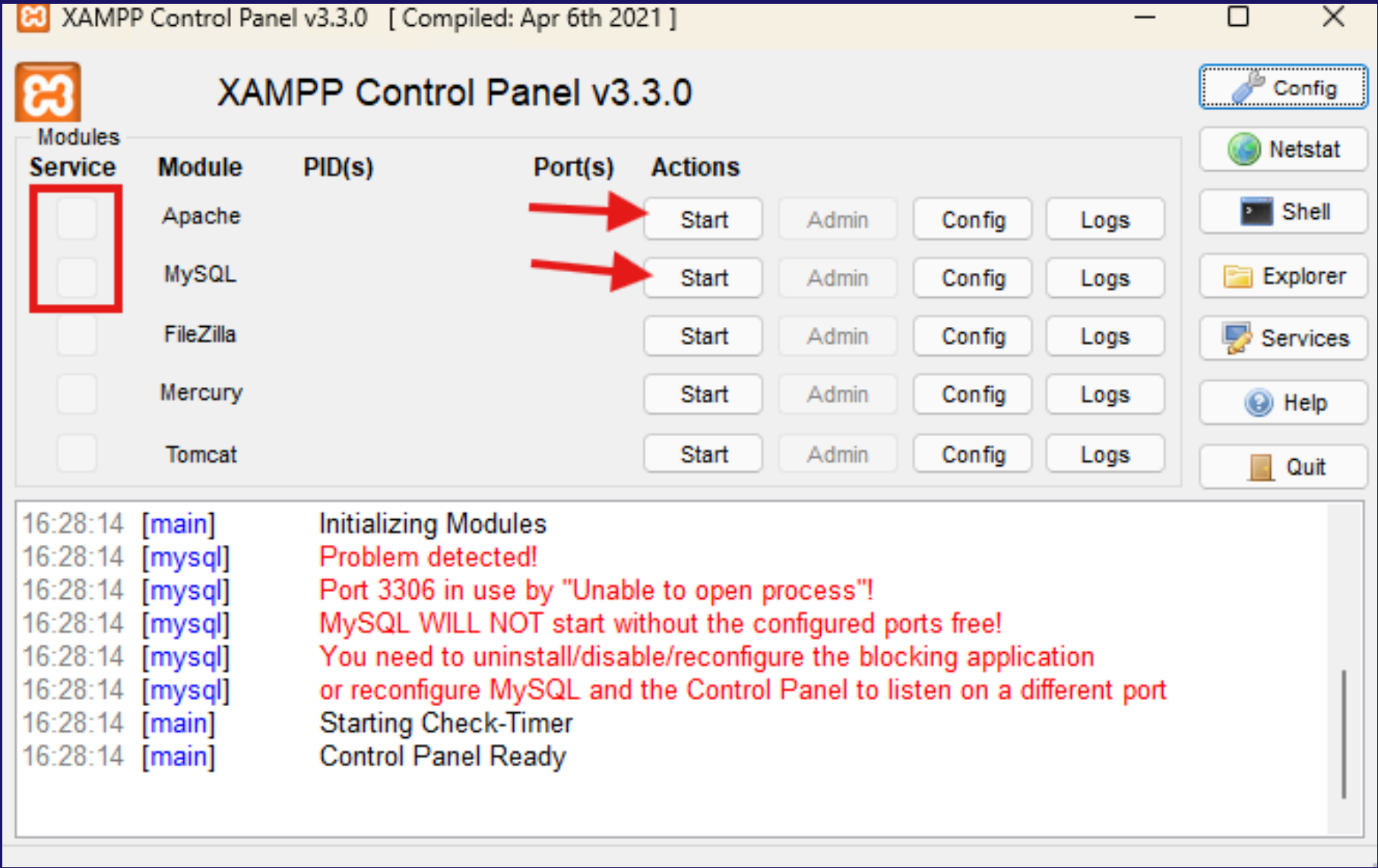
Baixar
Clique aqui para outras versões

 **XAMPP para Windows**
8.2.12 (PHP 8.2.12)

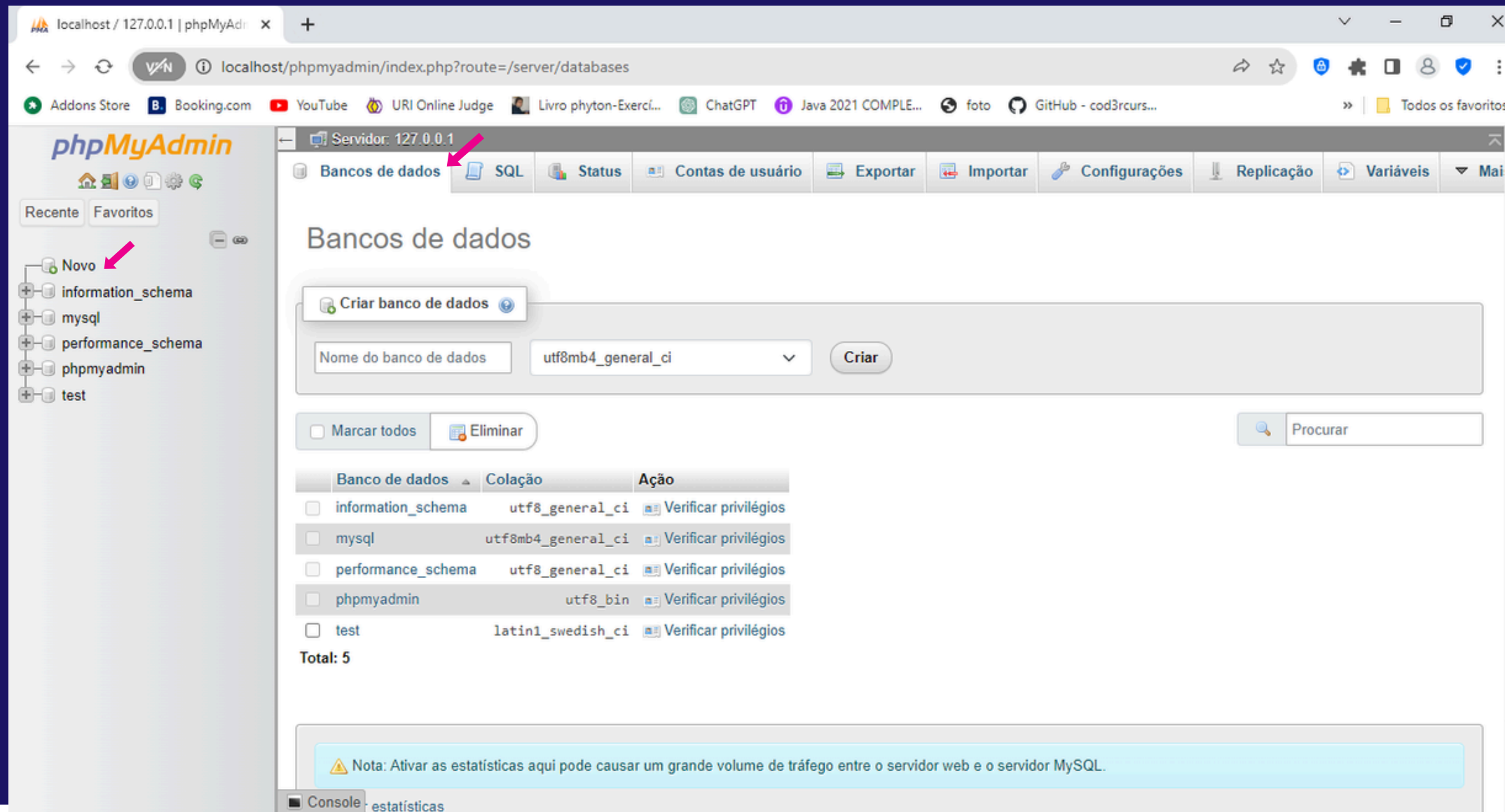
 **XAMPP para Linux**
8.2.12 (PHP 8.2.12)

 **XAMPP para OS X**
8.2.4 (PHP 8.2.4)

Xampp



PHPMyAdmin



CREATE

A instrução CREATE DATABASE, como o próprio nome sugere, serve para criarmos a base de dados na qual as tabelas serão criadas.

sintaxe:

```
CREATE DATABASE nome_do_banco;
```



CREATE

A instrução CREATE TABLE, O comando create table permite criarmos e definirmos a estrutura de uma tabela.

sintaxe:

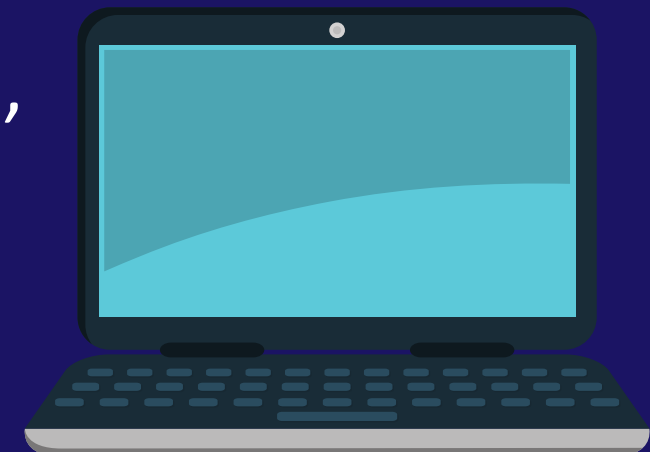
```
CREATE TABLE tabelas;
```

Os nomes das tabelas devem estar no plural.

Os nomes das colunas devem estar no singular.

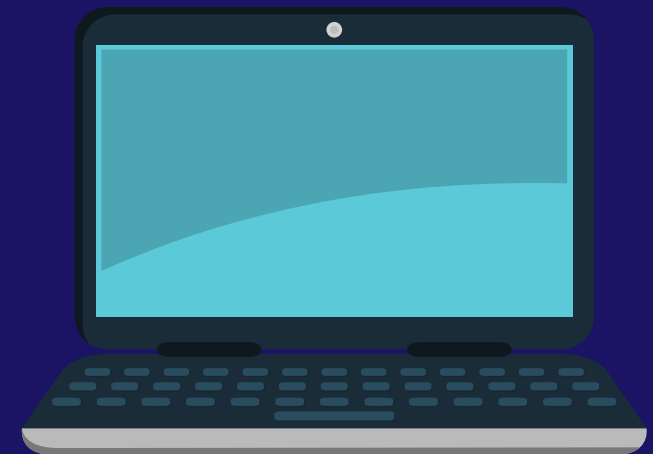
VAMOS PRATICAR!

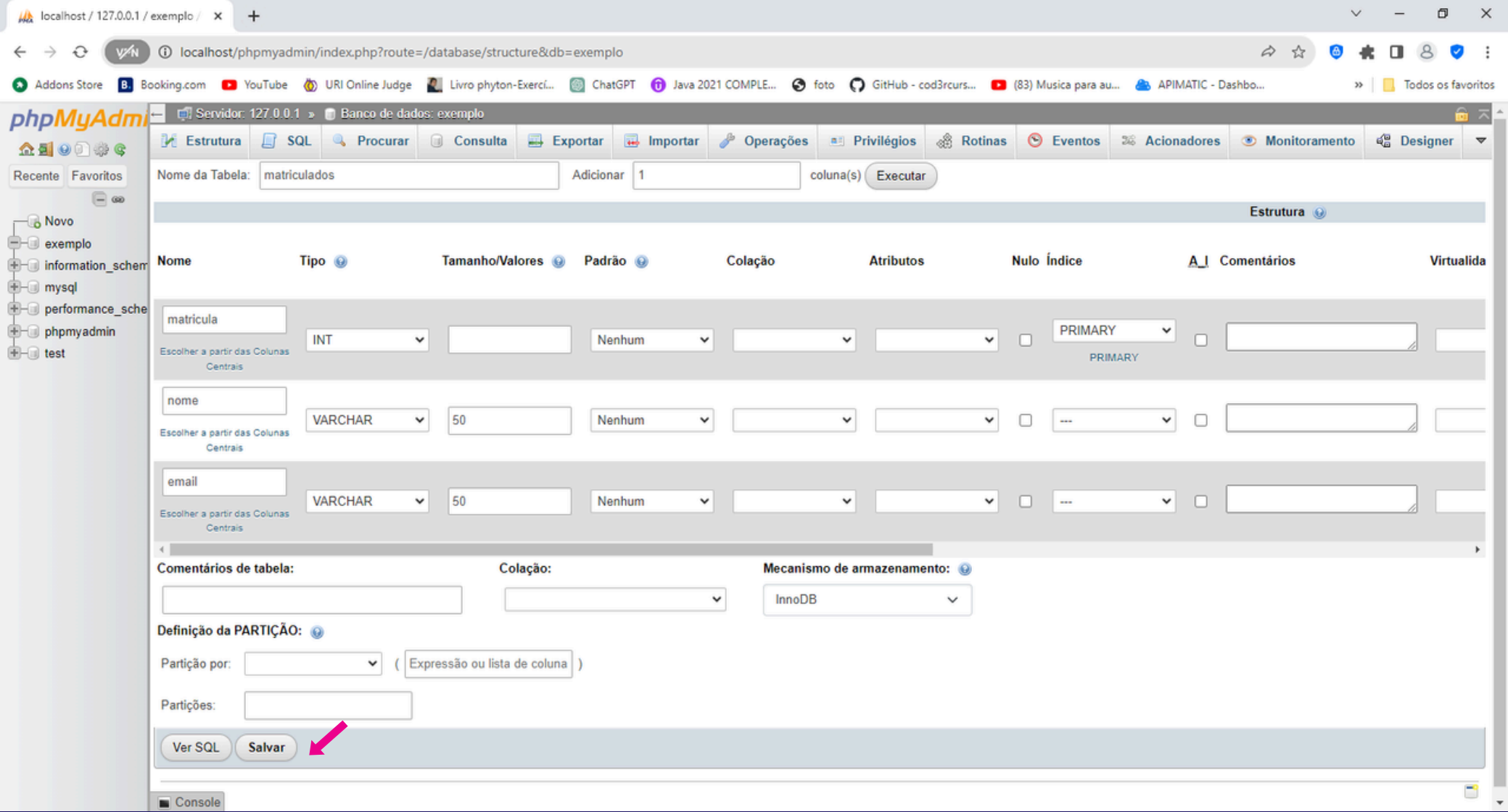
- Vamos criar o nosso primeiro banco de dados, com o nome: **alunos_secti**.



VAMOS PRATICAR!

- Agora, vamos criar uma tabela, com o nome: **matriculados**.
- Inserir 3 colunas:
- **id(cpf)**
- **nome**
- **email**





Nome da Tabela:

matriculados

Adicionar

coluna(s)

Ver SQL

Salvar

Executar

localhost / 127.0.0.1 / exemplo /

localhost/phpmyadmin/index.php?route=/database/structure&server=1&db=exemplo&table=matriculados

Addons StoreBooking.comYouTubeURI Online JudgeLivro python-Exerci...ChatGPTJava 2021 COMPLE...fotoGitHub - cod3rcurs...(83) Musica para au...APIMATIC - Dashbo...Todos os favoritos

phpMyAdmin

Servidor: 127.0.0.1 > Banco de dados: exemplo > Tabela: matriculados

VisualizarEstruturaSQLProcurarInserirExportarImportarPrivilégiosOperaçõesMonitoramentoAcionadores

RecenteFavoritos

Novo

exemplo

Nova

matriculados

information_schem

mysql

performance_sche

phpmyadmin

test

Estrutura da tabela

Visão de relação(ões)

#	Nome	Tipo	Colaço	Atributos	Nulo	Padrão	Comentários	Extra	Ação
☐	1 matrícula	int(11)			Não	Nenhum			Alterar Eliminar Mais
☐	2 nome	varchar(50)	utf8_general_ci		Não	Nenhum			Alterar Eliminar Mais
☐	3 email	varchar(50)	utf8_general_ci		Não	Nenhum			Alterar Eliminar Mais

☐ Marcar todos Com marcados: Visualizar Alterar Eliminar Primária Único Índice Espacial Texto completo Adicionar coluna(s) central(is)

Remover da(s) coluna(s) central(is)

ImprimirPropor estrutura de tabelaMonitorar tabelaMover campo(s)Normalizar

Adicionar 1 campo(s) após email Executar

Índices

Ação	Nome da chave	Tipo	Único	Pacote	Coluna	Cardinalidade	Colaço	Nulo	Comentário
Editar Renomear Eliminar	PRIMARY	BTREE	Sim	Não	matricula	0	A	Não	
Editar Renomear Eliminar	nome	BTREE	Sim	Não	nome	0	A	Não	

Criar um índice em 1 colunas Executar

Partições

Nenhum particionamento definido!

Tabela de partições

Console

localhost / 127.0.0.1 / exemplo / x +

localhost/phpmyadmin/index.php?route=/table/structure/save

phpMyAdmin

Servidor: 127.0.0.1 > Banco de dados: exemplo > Tabela: matriculados

Visualizar Estrutura SQL Procurar Inserir Exportar Importar Privilegios Operacoes Monitoramento Acionadores

Recente Favoritos

Novo exemplo Nova matriculados information_schema mysql performance_schema phpmyadmin test

✓ A tabela matriculados foi alterada com sucesso.

```
ALTER TABLE `matriculados` CHANGE `email` `email` VARCHAR(35) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL;
```

[Editar em linha] [Editar] [Criar código PHP]

Estrutura da tabela Visão de relação(ões)

#	Nome	Tipo	Colaço	Atributos	Nulo	Padrão	Comentários	Extra	Ação
☐	1 matrícula	int(11)			Não	Nenhum			Alterar Eliminar Mais
☐	2 nome	varchar(50)	utf8_general_ci		Não	Nenhum			Alterar Eliminar Mais
☐	3 email	varchar(35)	utf8_general_ci		Não	Nenhum			Alterar Eliminar Mais

↑ ☐ Marcar todos Com marcados: Visualizar Alterar Eliminar Primária Único Índice Espacial Texto completo Adicionar coluna(s) central(is)

Remover da(s) coluna(s) central(is)

Imprimir Propor estrutura de tabela Monitorar tabela Mover campo(s) Normalizar

Adicionar 1 campo(s) após email Executar

Índices

Ação	Nome da chave	Tipo	Único	Pacote	Coluna	Cardinalidade	Colaço	Nulo	Comentário
Editar Renomear Eliminar	PRIMARY	BTREE	Sim	Não	matricula	0	A	Não	
Editar Renomear Eliminar	nome	BTREE	Sim	Não	nome	0	A	Não	

Criar um índice em 1 colunas Executar

Console

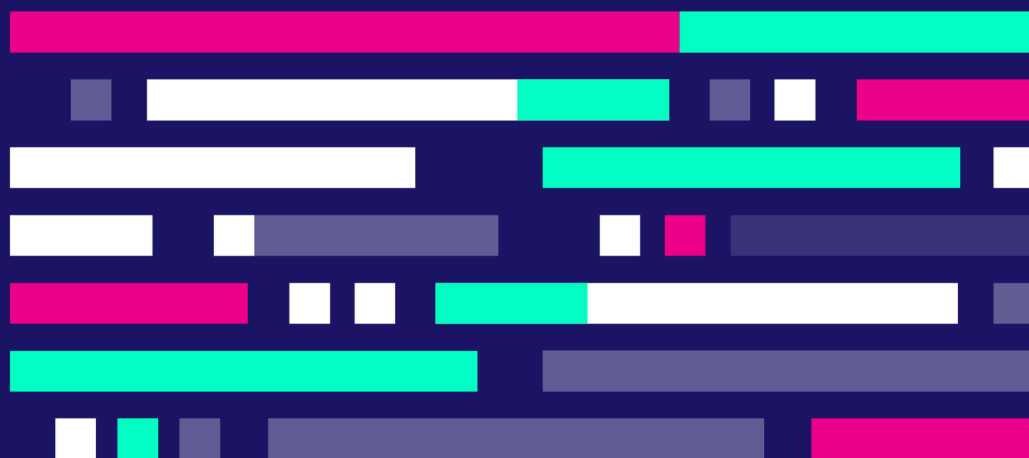
ALTER TABLE `matriculados` CHANGE `email` `email` VARCHAR(35) CHARACTER
SET utf8 COLLATE utf8_general_ci NOT NULL;

ALTER

A instrução ALTER TABLE permite alterar a estrutura de uma tabela acrescentando, alterando, retirando e alterando nomes, formatos das colunas.

sintaxe:

```
ALTER TABLE nomedatabela;  
DROP nomedacoluna;
```

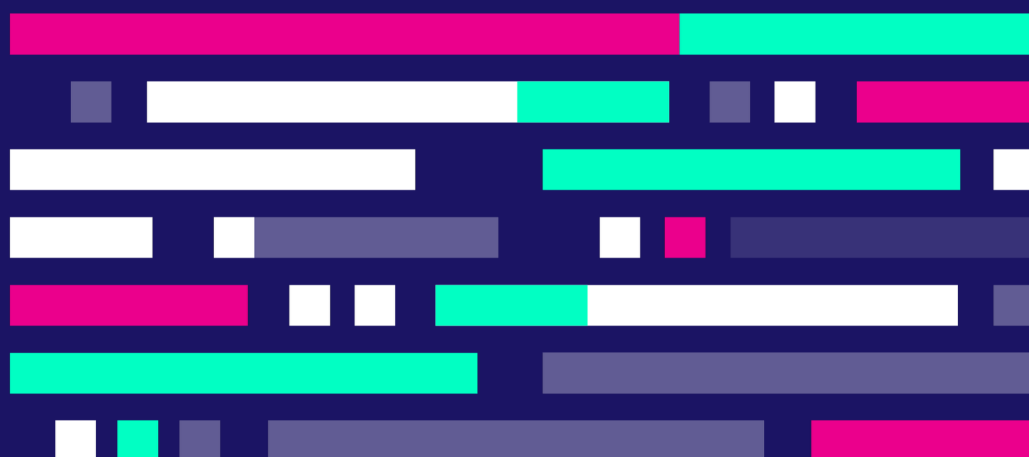


DROP

A instrução DROP. O comando drop table serve para destruímos uma tabela.

sintaxe:

```
DROP TABLE nomedatabela;
```

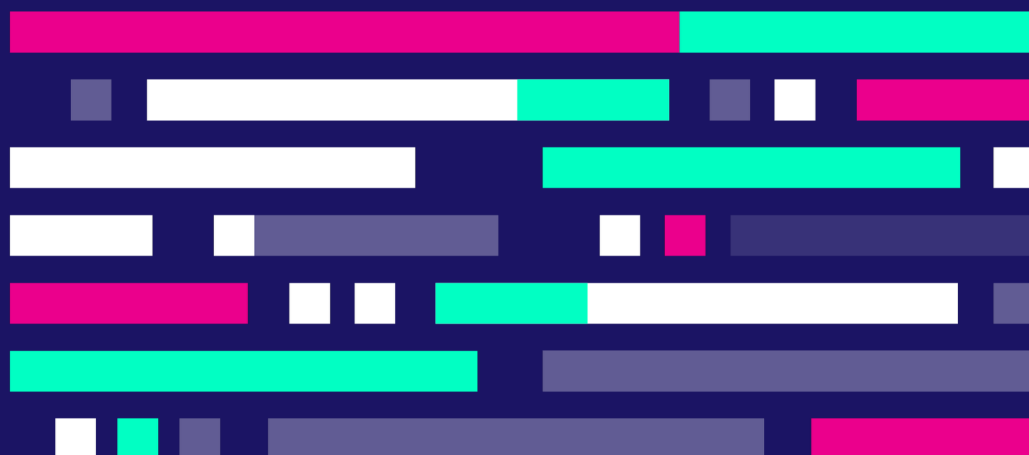


DROP

Já o parâmetro DROP serve para excluirmos um campo, um registro.

sintaxe:

```
ALTER TABLE + (NOME DA TABELA)  
DROP + campo ;
```



localhost / 127.0.0.1 / exemplo / x

localhost/phpmyadmin/index.php?route=/table/structure/save

Addons StoreBooking.comYouTubeURI Online JudgeLivro python-Exercí...ChatGPTJava 2021 COMPLE...fotoGitHub - cod3rcurs...(83) Musica para au...APIMATIC - Dashbo...Todos os favoritos

phpMyAdmin

Servidor: 127.0.0.1 » Banco de dados: exemplo » Tabela: matriculados

VisualizarEstruturaSQLProcurarInserirExportarImportarPrivilégiosOperaçõesMonitoramentoAcionadores

RecenteFavoritos

Novo

exemplo

Nova

matriculados

information_schem

mysql

performance_sche

phpmyadmin

test

A tabela matriculados foi alterada com sucesso.

ALTER TABLE `matriculados` ADD `dataNascimento` DATE NOT NULL AFTER `email`;

[Editar em linha] [Editar] [Criar código PHP]

Estrutura da tabelaVisão de relação(ões)

#	Nome	Tipo	Colaço	Atributos	Nulo	Padrão	Comentários	Extra	Ação
☐	1 matricula	int(11)			Não	Nenhum			AlterarEliminarMais
☐	2 nome	varchar(50)	utf8_general_ci		Não	Nenhum			AlterarEliminarMais
☐	3 email	varchar(35)	utf8_general_ci		Não	Nenhum			AlterarEliminarMais
☐	4 dataNascimento	date			Não	Nenhum			AlterarEliminarMais

Marcar todos

Com marcados:

Visualizar

Alterar

Remover da(s) coluna(s) central(is)

ÍndiceEspacialTexto completoAdicionar coluna(s) central(is)

ImprimirPropor estrutura de tabelaMonitorar tabelaMover campo(s)Normalizar

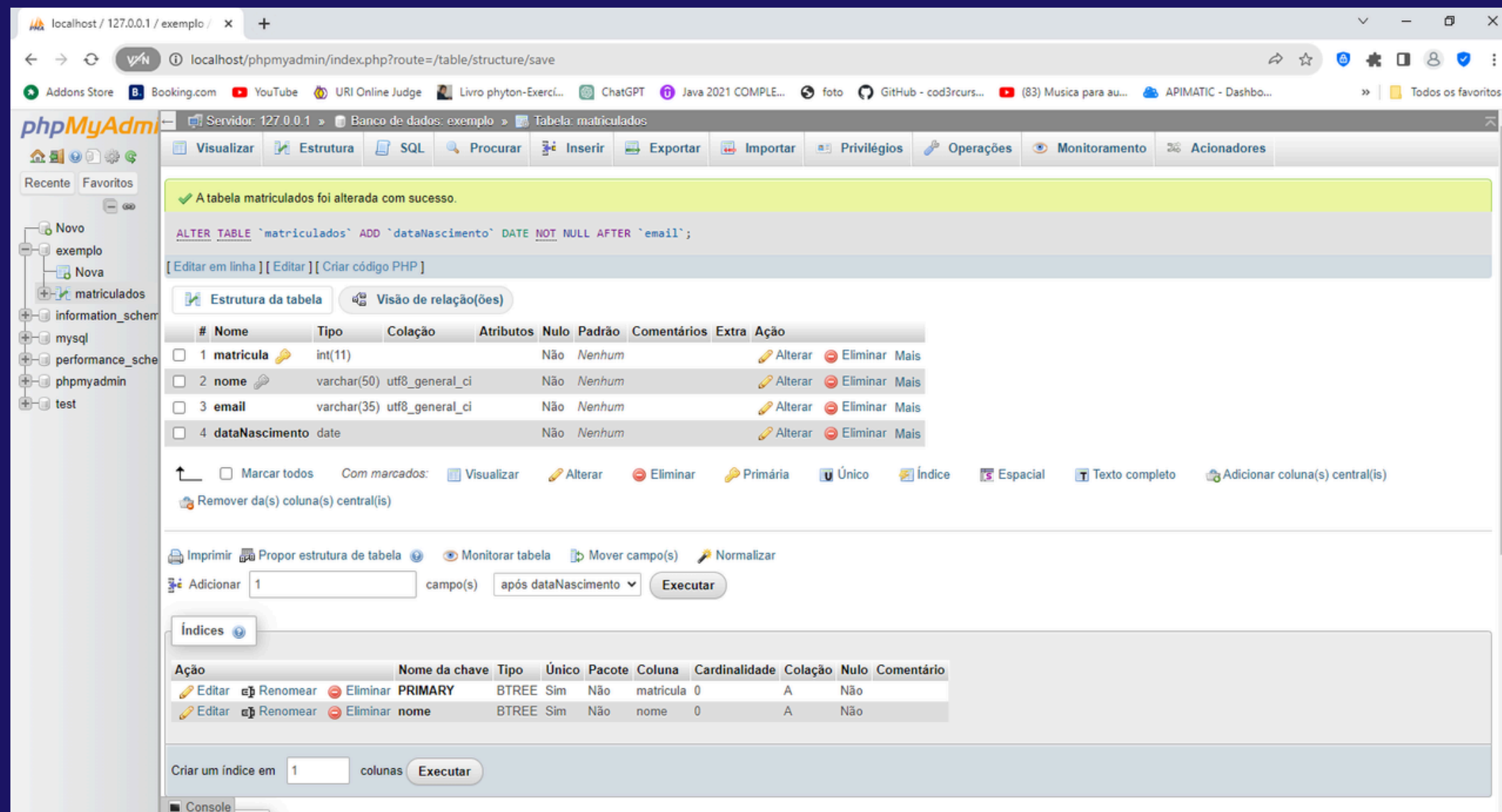
Adicionar1campo(s)após dataNascimentoExecutar

Índices

Ação	Nome da chave	Tipo	Único	Pacote	Coluna	Cardinalidade	Colaço	Nulo	Comentário
EditarRenomearEliminar	PRIMARY	BTREE	Sim	Não	matricula	0	A	Não	
EditarRenomearEliminar	nome	BTREE	Sim	Não	nome	0	A	Não	

Criar um índice em1colunasExecutar

Console



`ALTER TABLE `matriculados` ADD `dataNascimento` DATE NOT NULL AFTER `email`;`

AULA 3

Vamos lembrar..

Crie uma base de dados chamada **cadastro** e uma tabela chamada **pessoas** que contenha os seguintes campos:

- id
- nome
- nascimento
- sexo
- altura
- peso
- nacionalidade





Chaves

Em um banco de dados relacional, as chaves são utilizadas para identificar e relacionar registros em diferentes tabelas. Existem diferentes tipos de chaves, cada uma com sua finalidade específica.

As chaves em um banco de dados são elementos fundamentais para garantir a integridade, relacionamento e eficiência das operações de dados.

Identificação Única (Chave Primária)

Relacionamentos entre Tabelas (Chave Estrangeira).



Chave primária

- A chave primária é uma coluna ou conjunto de colunas em uma tabela que identifica de forma exclusiva cada registro nessa tabela.
- Ela não permite valores duplicados e geralmente é usada para vincular tabelas relacionadas.
- A chave primária é definida durante a criação da tabela usando a cláusula **PRIMARY KEY**.

```
CREATE TABLE clientes (  
    id INT PRIMARY KEY,  
    nome VARCHAR(30),  
);
```



Chave primária

Chave Primária

CodFunc	NomeFunc	Endereco	E-mail
01	João	Rua 123...	joao@teste.com
02	Pedro	Avenida ABC	pedro@teste.com
03	Maria	Rua etc...	maria@teste.com
04	José	Rua Teste...	jose@teste.com





Chave estrangeira

- A chave estrangeira é uma coluna que estabelece uma relação entre duas tabelas, referenciando a chave primária ou uma chave única em outra tabela.
- Ela mantém a integridade referencial entre as tabelas.
- A chave estrangeira é definida usando a cláusula FOREIGN KEY.

```
CREATE TABLE pedidos (  
    id_pedido INT PRIMARY KEY,  
    id_cliente INT,  
    FOREIGN KEY (id_cliente) REFERENCES  
    clientes(id_cliente)  
);
```

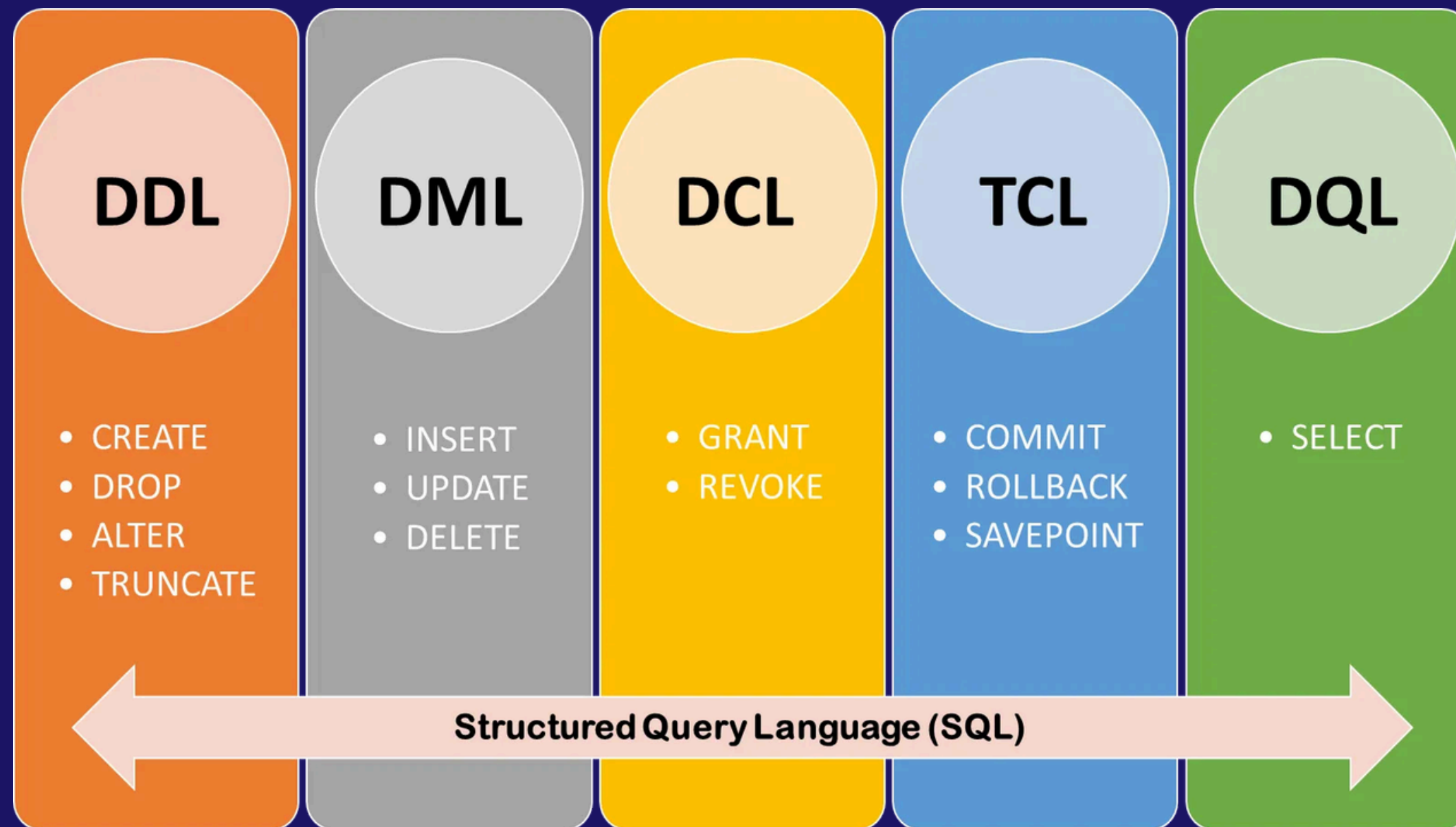


Chave estrangeira

<u>CodFunc</u>	NomeFunc	CodDep
01	João	D1
02	Pedro	D3
03	Maria	D1
04	José	D2

Chave Estrangeira

<u>CodDep</u>	Descricao
D1	Vendas
D2	Compras
D3	Manutenção





- **DDL - Linguagem de Definição de Dados** - Define os comandos utilizados para criação (CREATE) de tabelas, views, índices, atualização dessas estruturas (ALTER), assim como a remoção (DROP);
- **DML - Linguagem de Manipulação de Dados** - Define os comandos utilizados para manipulação de dados no banco (INSERT, UPDATE e DELETE);
- **DQL - Linguagem de Consulta de Dados** - Define o comando utilizado para que possamos consultar (SELECT) os dados armazenados no banco;



DML (Linguagem de Manipulação de dados)

A Linguagem de Manipulação de Dados é uma parte do SQL, que se concentra nas operações relacionadas à manipulação dos dados armazenados em um banco de dados. As principais operações DML incluem:
SELECT, INSERT, UPDATE e DELETE.

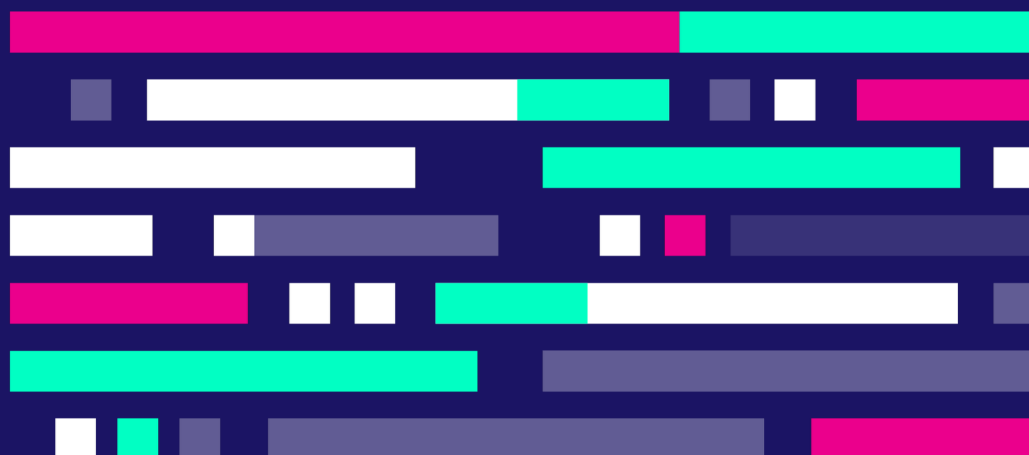


INSERT

Para incluirmos dados em uma tabela, utilizamos a instrução insert into. Este comando permite inserir um ou vários registros em uma tabela do banco de dados.

sintaxe:

```
INSERT INTO nome_da_tabela  
VALUES relacao_dos_valores;
```



INSERT

phpMyAdmin - Servidor: 127.0.0.1 » Banco de dados: akidauanabase » Tabela: alunos

Visualizar Estrutura SQL Procurar **Inserir** Exportar Importar Privilégios Operações Monitoramento Acionadores

Recente Favoritos

Novo akidauanabase Nova alunos Colunas Índices alunos_secti exemplo information_schem mysql performance_sche phpmyadmin test

Coluna	Tipo	Função	Nulo	Valor
nome	varchar(50)			Maria
matricula	int(11)			2321
email	varchar(30)			maria@maria.com.br
telefone	varchar(20)			+55 83 99234-3456

Executar

☐ Ignorar

Coluna	Tipo	Função	Nulo	Valor
nome	varchar(50)			João
matricula	int(11)			2124
email	varchar(30)			joao@joao.com.br
telefone	varchar(20)			+55 83 99234-3479

Console Favoritos Opções Histórico Limpar

```
INSERT INTO `alunos` (`nome`, `matricula`, `email`, `telefone`)
VALUES ('Maria', '2321', 'maria@maria.com.br', '+55 83 99234-3456'),
('João', '2124', 'joao@joao.com.br', '+55 83 99234-3479');
```

INSERT

Servidor: 127.0.0.1 » Banco de dados: akidauanabase » Tabela: alunos

Visualizar Estrutura SQL Procurar Inserir Exportar Importar Privilégios Operações Monitoramento Acionadores

Recente Favoritos

Novo
akidauanabase
Nova
alunos
Colunas
Índices
alunos_secti
exemplo
information_schem
mysql
performance_sche
phpmyadmin
test

Mostrando registros 0 - 1 (2 no total, Consulta levou 0.0005 segundos.)

`SELECT * FROM `alunos``

☐ Perfil [[Editar em linha](#)] [[Editar](#)] [[Demonstrar SQL](#)] [[Criar código PHP](#)] [[Atualizar](#)]

☐ Mostrar tudo | Número de linhas: 25 | Filtrar linhas: Procurar nesta tabela | Ordenar pela chave: Nenhum

Opções extras

	nome	matricula	email	telefone
☐ Editar Copiar Remover	João	2124	joao@joao.com.br	+55 83 99234-3479
☐ Editar Copiar Remover	Maria	2321	maria@maria.com.br	+55 83 99234-3456

☐ Marcar todos Com marcados: [Editar](#) [Copiar](#) [Remover](#) [Exportar](#)

☐ Mostrar tudo | Número de linhas: 25 | Filtrar linhas: Procurar nesta tabela | Ordenar pela chave: Nenhum

Operações resultantes das consultas

[Imprimir](#) [Copiar para a área de transferência](#) [Exportar](#) [Exibir gráfico](#) [Criar view](#)

[Marcar essa consulta SQL](#)

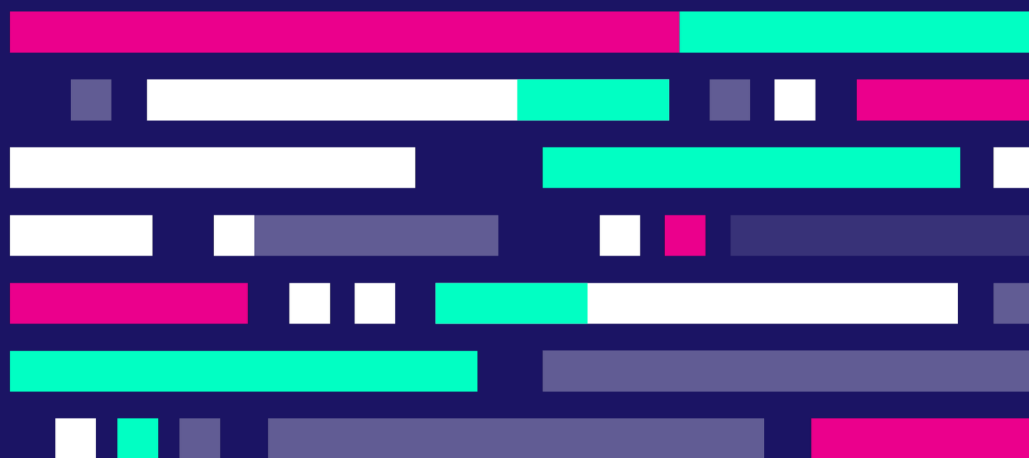
localhost/phpmyadmin/index.php?route=/sql&db=akidauanabase&table=alunos... Favoritos Opções Histórico Limpar

UPDATE

Para alterarmos uma informação contida numa tabela do banco de dados, utilizamos o comando update.

sintaxe:

```
UPDATE nome_da_tabela  
    SET nome_coluna = novo_nome  
    WHERE condição;
```



UPDATE

The screenshot shows the phpMyAdmin interface with the 'alunos' table selected. The top navigation bar includes options like Visualizar, Estrutura, SQL, Procurar, Inserir, Exportar, Importar, Privilégios, Operações, Monitoramento, and Acionadores. The left sidebar shows the database structure with 'akidauanabase' and its tables: 'alunos', 'alunos_secti', 'exemplo', 'information_schem', 'mysql', 'performance_sche', 'phpmyadmin', and 'test'.

The main content area displays the execution of an UPDATE query:

```
UPDATE `alunos` SET `telefone` = '+55 83 99234-3567' WHERE `alunos`.`matricula` = 2124;
```

A confirmation message states: "1 linha afetada." (1 line affected).

Below the query, a SELECT query is shown, displaying the results of the update:

```
SELECT * FROM `alunos`
```

The results are shown in a table with columns: nome, matricula, email, and telefone. Two rows are displayed:

	nome	matricula	email	telefone
☐	João	2124	joao@joao.com.br	+55 83 99234-3567
☐	Maria	2321	maria@maria.com.br	+55 83 99234-3456

At the bottom, there are controls for filtering and sorting, including a search bar and a dropdown menu for sorting by key (Nenhum).

UPDATE `alunos`
SET `telefone` = '+55 83 99234-3567'
' WHERE `alunos`.`matricula` = 2124;

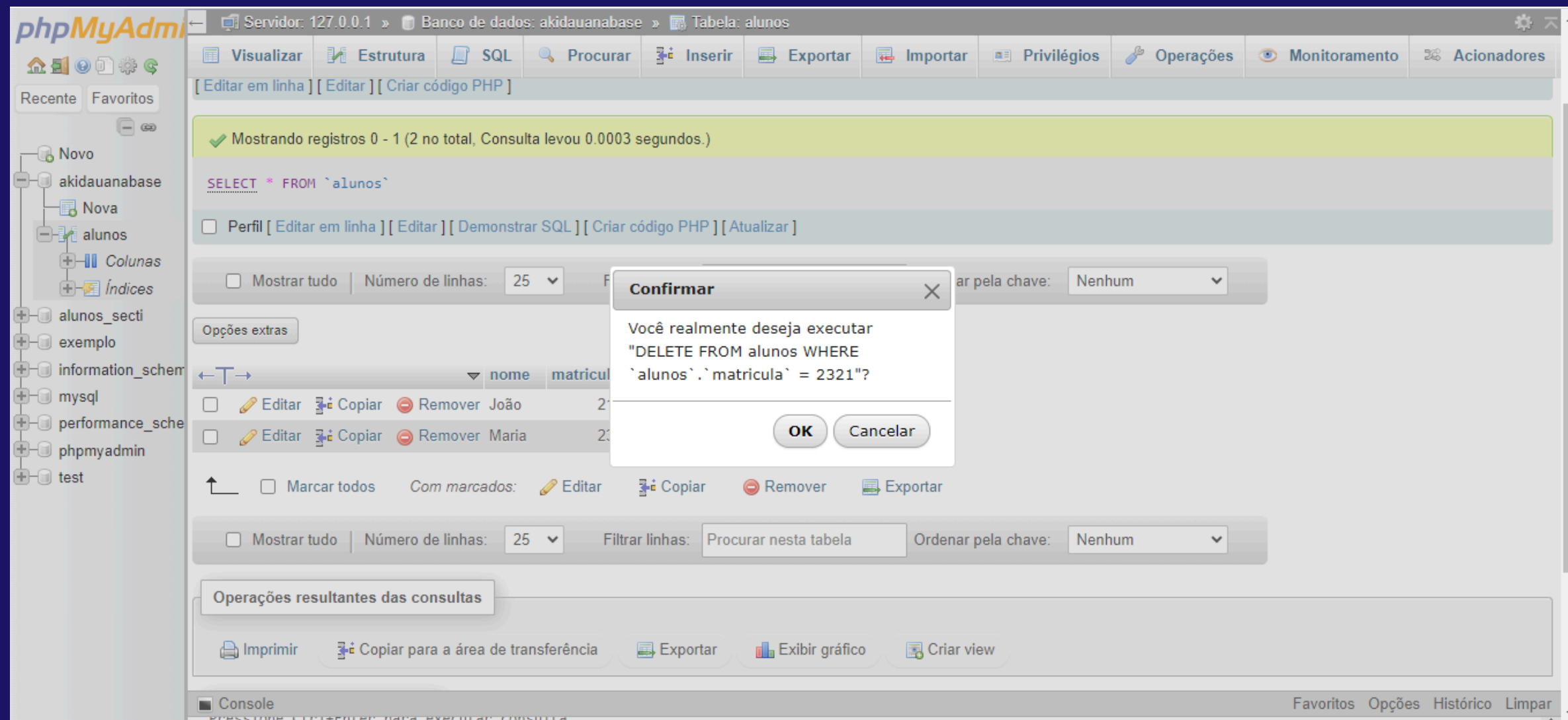
DELETE

O comando delete é utilizado para excluir linhas de uma tabela

sintaxe:

```
DELETE FROM nome_da_tabela  
WHERE condicao;
```





DELETE FROM `alunos`
WHERE `alunos`.`matricula` =2321;

AULA 4

Consultas simples

Consultar dados em um banco de dados, normalmente, é a operação mais utilizada pelos usuários. Para isso, precisamos fazer uso da instrução select. Ela é considerada por muitos, como a instrução mais poderosa da linguagem SQL.

A instrução SELECT no SQL é utilizada para recuperar dados de um ou mais tabelas em um banco de dados relacional. Essa instrução é fundamental para a realização de consultas, permitindo que você especifique quais dados deseja recuperar e sob quais condições.

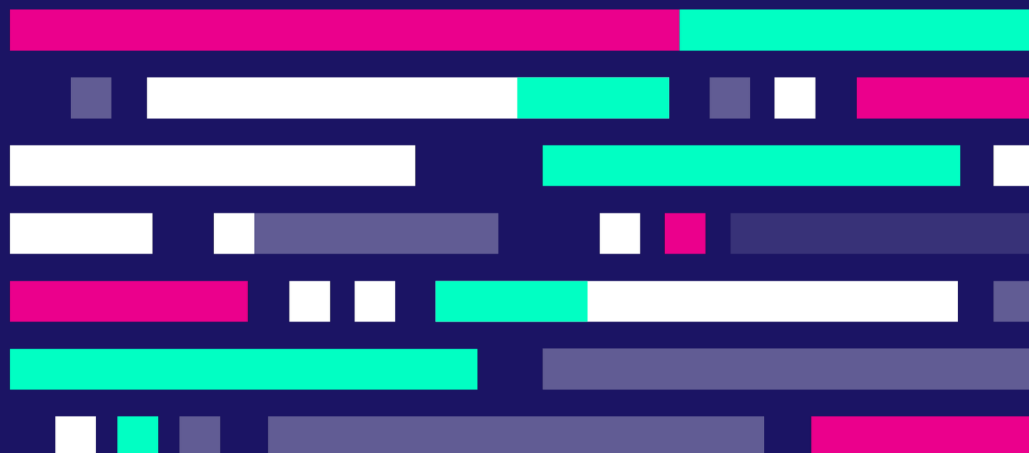


SELECT

Obter todas as informações de uma tabela específica.

sintaxe:

```
SELECT * FROM nome_da_tabela;
```



WHERE

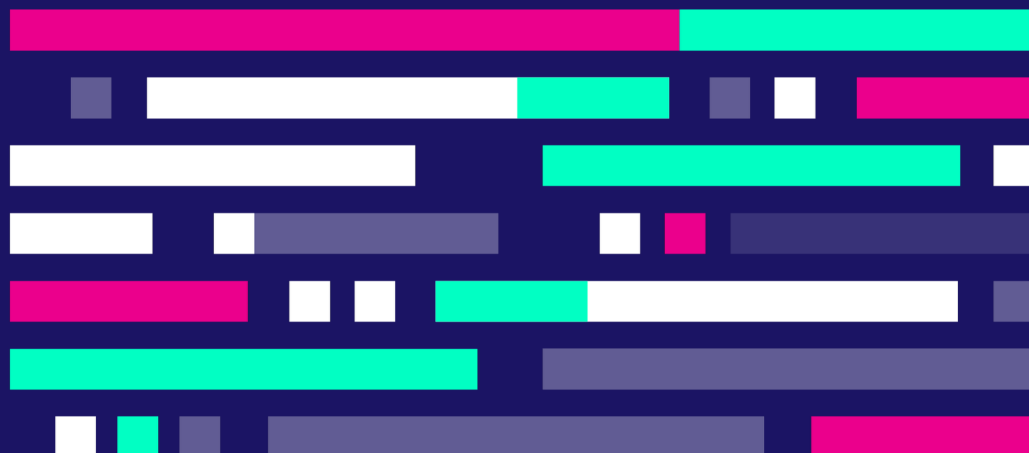
Define os filtros que serão aplicados na consulta.

sintaxe:

```
SELECT * FROM `pessoas` WHERE nome = 'Luanna';
```



O WHERE vem de "onde".
Vamos especificar onde alguma
informação específica seja tal
coisa.



DISTINCT

A palavra-chave **DISTINCT** em SQL é usada em consultas para retornar apenas valores únicos em uma coluna ou conjunto de colunas. Isso é útil quando você deseja eliminar duplicatas e obter apenas valores distintos.

sintaxe:

```
SELECT DISTINCT cliente FROM  
`tabela_pedidos`;
```

DISTINCT

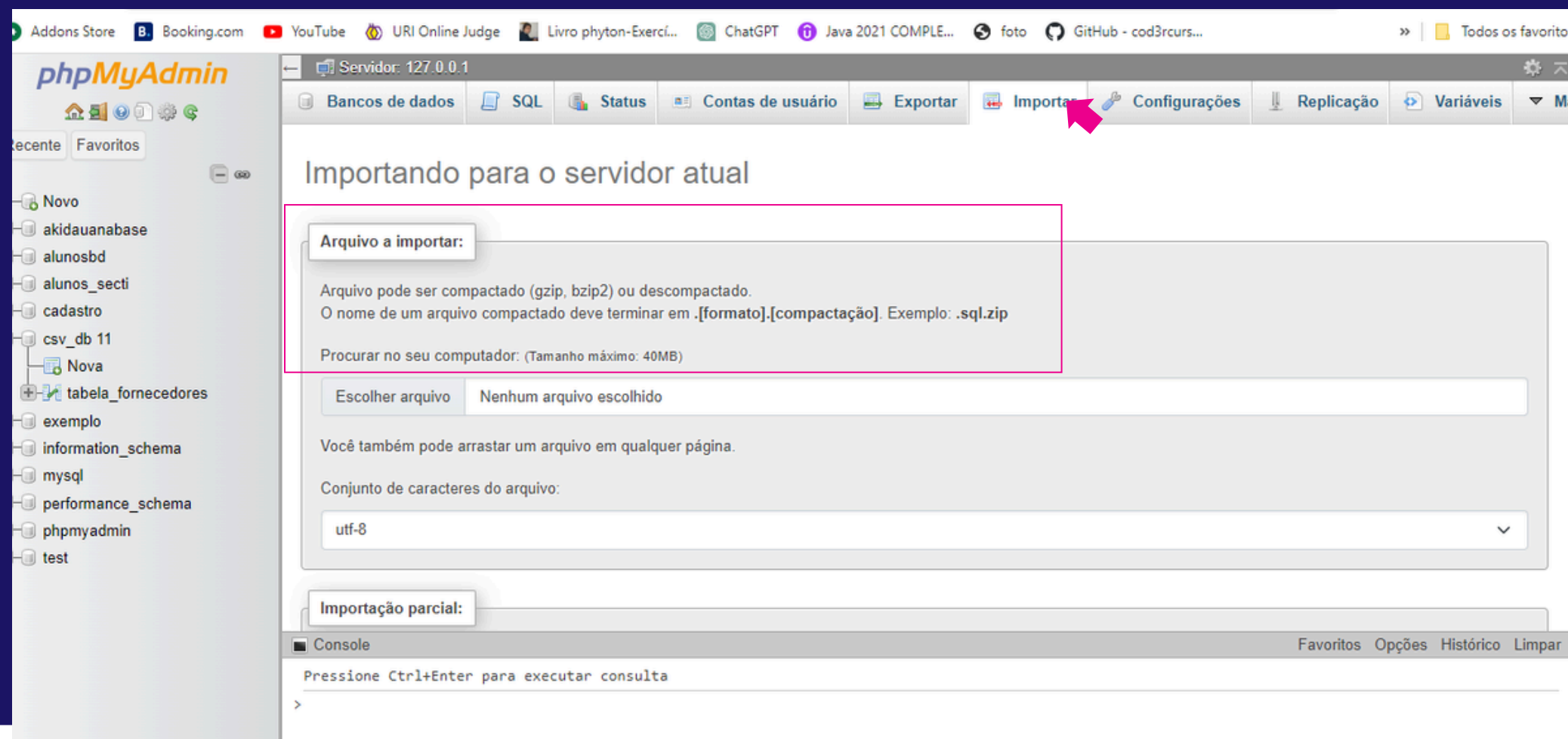
Exemplo: obter uma lista de cidades distintas dos clientes.

```
SELECT DISTINCT cidade  
FROM clientes;
```

Este exemplo retornará uma lista de cidades únicas da tabela clientes. A palavra-chave DISTINCT garante que cada cidade seja listada apenas uma vez, eliminando duplicatas.



Como importar uma Tabela já existente para o PHPMyAdmin??



Vamos praticar..

- A partir da base de dados 'tabela_fornecedores' presente no drive da turma, transforme os dados para que seja possível trabalhar com a tabela no PHPMysqlAdmin.

Por exemplo: Modifique os nomes das colunas, Determine a chave primária dessa tabela, exclua linhas em branco.

- Será que nós conseguiríamos filtrar somente os fornecedores da China?





Resposta:

```
SELECT * FROM `tabela_fornecedores` WHERE Pais_Origem = 'China';
```



Vamos praticar..

- Utilizando comandos, criar a tabela 'tabelaclientes' com os seguintes campos:
id_cliente (chave primária)
nome_cliente
contato_cliente
- Alterar a 'tabelaclientes' adicionando o campo 'endereco_cliente'.
- Adicionar novos clientes base de dados, através do comando INSERT.



Respostas:

```
create table `tabela_clientes` (`id_cliente` INT PRIMARY KEY , `nome_cliente`  
VARCHAR (80), `contato_cliente` VARCHAR (30));
```

```
ALTER TABLE `tabela_clientes` ADD `endereco_cliente` varchar (200);
```



Vamos praticar...

Criar uma tabela 'tabela_categorias' com os seguintes campos:

id_categoria (chave primária)

nome_categoria

descricao_categoria



Resposta:

```
Create table `tabela_categorias` (`id_categoria` INT PRIMARY KEY,  
`nome_categoria` VARCHAR (200), `descricao_categoria` TEXT);
```



AULA 5

Relacionamento entre tabelas

A chave estrangeira é responsável por relacionar as tabelas, sendo a coluna que faz a ligação entre duas tabelas diferentes.

Exemplo: Temos a tabela de pedidos e, dentro dela, há uma coluna chamada Cliente. Essa coluna contém o ID dos clientes, mas não todas as informações deles, apenas os identificadores, e esses IDs estão ligados à tabela de clientes. Portanto, a coluna Cliente é uma chave estrangeira que vem da tabela de clientes.



Vamos praticar...

A chave estrangeira criará uma ligação entre duas tabelas, ou seja, é a coluna pela qual essas tabelas vão se relacionar, se conversar.

Exemplo: Temos a tabela de pedidos e, dentro dela, há uma coluna chamada Cliente. Essa coluna contém o ID dos clientes, mas não todas as informações deles, apenas os identificadores, e esses IDs estão ligados à tabela de clientes. Portanto, a coluna Cliente é uma chave estrangeira que vem da tabela de clientes.



Vamos praticar..

Criar a tabela 'tabela_produtos' com os seguintes campos:

id_produto (chave primária)

nome_produto

descricao_produto

categoria

preco_de_compra DECIMAL (5,2)

unidade_produto

fornecedor INT (chave estrangeira)

data_inclusao



Resposta:

```
CREATE TABLE tabela_produtos (  
    id_produto INT NOT NULL AUTO_INCREMENT,  
    nome VARCHAR(100) NOT NULL,  
    descricao TEXT,  
    categoria VARCHAR(50),  
    preco_de_compra DECIMAL(10,2) NOT NULL,  
    unidade VARCHAR(20),  
    fornecedor INT NOT NULL,  
    data_inclusao DATE DEFAULT (CURRENT_DATE),  
    PRIMARY KEY (id_produto),  
    CONSTRAINT fk_fornecedor  
        FOREIGN KEY (fornecedor) REFERENCES fornecedores(id_fornecedor)  
) ENGINE=InnoDB;
```



Vamos praticar... Agora crie essa tabela:

Criar a tabela 'tabela_fornecedor' com os seguintes campos:

id_fornecedor (chave primária),

nome_fornecedor,

cnpj,

telefone,

email,

endereco,

data_cadastro



Resposta:

```
CREATE TABLE tabela_fornecedor (  
  id_fornecedor INT NOT NULL AUTO_INCREMENT,  
  nome_fornecedor VARCHAR(100) NOT NULL,  
  cnpj VARCHAR(18) UNIQUE NOT NULL,  
  telefone VARCHAR(15),  
  email VARCHAR(100),  
  endereco VARCHAR(255),  
  data_cadastro DATE DEFAULT (CURRENT_DATE),  
  PRIMARY KEY (id_fornecedor)  
);
```



Sintaxe - INSERT

Sintaxe:

```
INSERT INTO nome_tabela (colunas_da_tabela)  
VALUES (valor1, valor2, valor3...),
```



Inserindo dados - INSERT INTO

```
INSERT INTO tabela_fornecedores  
(nome_fornecedor, cnpj, telefone, email, endereco)  
VALUES (  
    'Fornecedor A',  
    '12.345.678/0001-90',  
    '(11) 99999-0001',  
    'contato@fornecedorA.com',  
    'Rua A, 100');
```



Por fim, vamos criar a tabela pedido:

Criar a tabela 'tabela_pedido' com os seguintes campos:

id_pedido (chave primária),
data_pedido,
id_produto,
quantidade,
preco_unitario,
total,
status_pedido,
fk_pedido (chave estrangeira)



Resposta:

```
CREATE TABLE tabela_pedidos (  
  id_pedido INT NOT NULL AUTO_INCREMENT,  
  data_pedido DATE DEFAULT (CURRENT_DATE),  
  id_produto INT NOT NULL,  
  quantidade INT NOT NULL,  
  preco_unitario DECIMAL(10,2) NOT NULL,  
  total DECIMAL(10,2) GENERATED ALWAYS AS (quantidade * preco_unitario) STORED,  
  status_pedido VARCHAR(50) NOT NULL,  
  PRIMARY KEY (id_pedido),  
  CONSTRAINT fk_pedido_produto  
    FOREIGN KEY (id_produto) REFERENCES produtos(id_produto)
```



Alterando os dados - UPDATE

Mas e se quisermos alterar algum dado específico no banco de dados de forma definitiva é possível?

Sim, conseguimos fazer isso usando a cláusula UPDATE.
É isso que entenderemos na prática.

Sintaxe:

```
UPDATE nome_tabela
```

```
SET coluna1 = novo_valor, coluna2 = novo_valor ...
```

```
WHERE condição;
```



Vamos praticar...

Precisamos alterar o status de "processando" para "enviado", pois os pedidos já foram enviados para os clientes.

Resposta:

```
UPDATE `tabela_pedidos` SET status = "Enviado" WHERE id_pedido = 1;
```



Deletando Dados - DELETE

A cláusula DELETE vem sempre acompanhada do WHERE para especificar onde e o que será deletado. Tome muito cuidado ao fazer essas alterações, pois é difícil desfaze-las.

Vamos praticar...

Deletar da “tabela_fornecedores” os fornecedores com país de origem Turquia.



Resposta:

```
DELETE FROM `tabela_fornecedores` WHERE  
pais_origem = "Turquia";
```



se quiséssemos deletar todos os fornecedores com ID maior que 35. Sabendo que cancelamos relações comerciais com todos esses fornecedores, como poderíamos deletar tudo de uma vez?

```
DELETE FROM `tabela_fornecedores` WHERE id_fornecedor > 35;
```



AULA 6-7

Operadores lógicos matemáticos

Uma das formas de filtrar os nossos dados, ou seja, trazer resultados específicos, é através dos operadores lógico-matemáticos. Vamos estudar como eles podem nos ajudar a filtrar as informações de maneiras diferentes, trazendo exatamente o que queremos observar nos nossos dados.

	Operador		
>	Maior que	$x > y$	x é maior que y?
>=	Maior ou igual	$x \geq y$	x é maior ou igual a y ?
<	Menor que	$x < y$	x é menor que y?
<=	Menor ou igual	$x \leq y$	x é menor ou igual a y ?
==	Igualdade	$x == y$	x é igual a y?

Aplicando Filtros- Maior, menor, igual

Ex: Queremos filtrar, para a nossa empresa, os pedidos cujo valor total seja maior que R\$ 200. Como podemos fazer?

```
SELECT * FROM `tabela_pedidos` WHERE total_do_pedido > 200
```

				ID	Data_do_pedido	Status	Total_do_pedido	Cliente	Data_envio_estimada
☐	 Editar	 Copiar	 Remover	10	2023-08-10	Enviado	420.75	10	2023-08-16
☐	 Editar	 Copiar	 Remover	12	2023-08-12	Enviado	320.25	7	2023-08-19
☐	 Editar	 Copiar	 Remover	14	2023-08-14	Entregue	420.50	11	2023-08-25
☐	 Editar	 Copiar	 Remover	15	2023-08-15	Enviado	300.00	13	2023-08-17
☐	 Editar	 Copiar	 Remover	16	2023-08-16	Enviado	510.25	16	2023-08-21
☐	 Editar	 Copiar	 Remover	18	2023-08-18	Enviado	420.90	18	2023-08-13
☐	 Editar	 Copiar	 Remover	19	2023-08-19	Enviado	720.25	20	2023-08-23



Filtrando strings

EX: Como poderíamos utilizar os operadores lógicos para filtrar strings? Se a nossa empresa quisesse filtrar só os clientes que têm como inicial do nome a letra C para frente, excluindo aqueles com inicial A ou B, poderíamos utilizar a mesma base lógica.

Tudo do seu portfolio

```
SELECT * FROM `tabela_fornecedores` WHERE Nome_Fornecedor > "C"
```





Filtrando Datas

A mesma lógica se aplica para filtrar datas usando os operadores lógicos. A empresa quer saber quais pedidos foram feitos depois do dia 19 de setembro.

```
SELECT * FROM `tabela_pedidos` WHERE Data_do_pedido > "2023-09-19";
```

Tudo do seu pedido





Filtros Compostos- AND, OR, NOT e BETWEEN

Consideremos o seguinte exemplo: Se a empresa não quer saber apenas os pedidos que têm valor maior ou igual a 200 reais. Ela quer, além de saber se esses pedidos têm esse valor, que eles sejam os pedidos com status pendente, ou seja, que não foram enviados ainda.

Então, como nós poderíamos consultar essas duas informações ao mesmo tempo?
Por meio dos filtros compostos!



AND

AND - conjunção “e”. Usado para uma nova especificação.

Ex: Vamos especificar que a coluna Status deve ser igual a 'Pendente'.

SELECT * FROM `tabela\_pedidos` WHERE Total_do_pedido >= 200 AND Status = "Pendente"

				ID	Data_do_pedido	Status	Total_do_pedido	Cliente	Data_envio_estimada
☐	 Editar	 Copiar	 Remover	22	2023-08-22	Pendente	250.00	21	2023-08-30
☐	 Editar	 Copiar	 Remover	30	2023-08-30	Pendente	210.50	27	2023-09-04
☐	 Editar	 Copiar	 Remover	42	2023-09-11	Pendente	250.50	42	2023-09-19
☐	 Editar	 Copiar	 Remover	50	2023-09-19	Pendente	200.50	46	2023-09-27
☐	 Editar	 Copiar	 Remover	55	2023-09-24	Pendente	520.00	3	2023-10-05
☐	 Editar	 Copiar	 Remover	58	2023-09-27	Pendente	420.75	5	2023-10-06

OR

OR-“ou”. Podemos utilizar uma especificação de "ou uma coisa ou outra" para trazer resultados diferentes.

Ex: Consultar todos os pedidos que não foram enviados ainda, ou seja, os pedidos que estejam com status ou Pendente ou Processando.

```
SELECT * FROM `tabela_pedidos` WHERE Status = "Pendente" OR Status = "enviado";
```

Government	Percentage
Current government	85%
Previous government	15%

NOT

NOT-“NÃO”. NOT é de negação. Ou seja, utilizada para fazer um filtro por exclusão e não por inclusão.

Ex: Consultar todos os pedidos que não têm o status Pendente.

```
SELECT * FROM `tabela_pedidos` WHERE NOT Status = "Pendente";
```

				ID	Data_do_pedido	Status	Total_do_pedido	Cliente	Data_envio_estimada			
☐		Editar		Copiar		Remover	10	2023-08-10	Enviado	420.75	10	2023-08-16
☐		Editar		Copiar		Remover	12	2023-08-12	Enviado	320.25	7	2023-08-19
☐		Editar		Copiar		Remover	14	2023-08-14	Entregue	420.50	11	2023-08-25
☐		Editar		Copiar		Remover	15	2023-08-15	Enviado	300.00	13	2023-08-17
☐		Editar		Copiar		Remover	16	2023-08-16	Enviado	510.25	16	2023-08-21

BETWEEN

BETWEEN-“ENTRE”. Podemos utilizar para especificar um intervalo de dados. Bastante usado para filtrar datas.

Ex: Filtrar pedidos enviados entre uma data e outra, por exemplo.

```
SELECT * FROM `tabela_pedidos` WHERE Data_envio_estimada BETWEEN "2023-08-01" AND "2023-09-01";
```

				ID	Data_do_pedido	Status	Total_do_pedido	Cliente	Data_envio_estimada	
☐		Editar	 Copiar	 Remover	1	2023-08-01	Pendente	150.00	5	2023-08-10
☐		Editar	 Copiar	 Remover	10	2023-08-10	Enviado	420.75	10	2023-08-16
☐		Editar	 Copiar	 Remover	11	2023-08-11	Pendente	180.00	1	2023-08-22
☐		Editar	 Copiar	 Remover	12	2023-08-12	Enviado	320.25	7	2023-08-19
☐		Editar	 Copiar	 Remover	13	2023-08-13	Pendente	90.75	3	2023-08-07
☐		Editar	 Copiar	 Remover	14	2023-08-14	Entregue	420.50	11	2023-08-25

AULA 8

ORDER BY

O comando ORDER BY é uma funcionalidade fundamental na linguagem SQL para classificar os resultados de uma consulta em uma ordem específica. Isso é útil quando queremos apresentar os dados de maneira organizada.

- ASC (Padrão): Ordem ascendente (do menor para o maior).
- DESC: Ordem descendente (do maior para o menor).



Vamos praticar....

EX:

```
SELECT * FROM `tabela_pedidos` WHERE Data_envio_estimada BETWEEN "2023-08-01" AND "2023-09-01" ORDER BY id;
```

```
SELECT * FROM `tabela_pedidos` WHERE total_do_pedido BETWEEN "200" AND "600" ORDER BY id;
```



ALIAS

ALIAS é um nome alternativo que você pode atribuir a uma tabela ou a uma coluna em uma consulta. São frequentemente usados para tornar o código mais legível e para simplificar consultas complexas. Eles são especialmente úteis quando você está manipulando múltiplas tabelas ou quando precisa renomear o resultado de uma expressão ou função.



Exemplo de uso do AS

Ex: Digamos que temos uma consulta que calcula a soma do valor total de pedidos e queremos dar um nome mais descritivo à coluna de resultado:

```
SELECT SUM(Total_do_pedido) AS total_vendas FROM `tabela_pedidos`
```

Neste exemplo, usamos AS total_de_vendas para atribuir o alias total_de_vendas à coluna resultante da soma do valor total dos pedidos. Isso torna o resultado mais claro e legível.

id_pedido	total_pedido
1	100.00
2	200.00
3	300.00
4	400.00
5	500.00
6	600.00
7	700.00
8	800.00
9	900.00
10	1000.00

Opções extras
total_vendas
21870.45

GROUP BY

O GROUP BY é utilizado principalmente com funções de agregação (como COUNT, SUM, AVG, MIN, MAX, etc.) para agrupar linhas que possuem valores semelhantes em uma ou mais colunas e permitir a aplicação dessas funções em cada grupo.

```
SELECT YEAR(Data_do_pedido) AS ano, SUM(Total_do_pedido) AS  
total_vendas FROM `tabela_pedidos` GROUP BY YEAR(Data_do_pedido)
```



```
SELECT YEAR(Data_do_pedido) AS ano, SUM(Total_do_pedido) AS total_vendas  
FROM `tabela_pedidos` GROUP BY YEAR(Data_do_pedido)
```

Neste exemplo:

- YEAR(Data_do_pedido) extrai o ano da data de cada pedido.
- SUM(Total_do_pedido) calcula o total de vendas para cada grupo de anos.
- GROUP BY YEAR(Data_do_pedido) agrupa as linhas pela parte do ano da data do pedido.

Isso nos dará uma saída com duas colunas: ano e total_vendas, onde cada linha representa o total de vendas para um ano específico.

ano	total_vendas
0	0
2023	21870.45



Além disso, podemos usar múltiplas colunas no GROUP BY. Por exemplo, se quisermos saber o total de vendas por ano e por mês:

```
SELECT YEAR(Data_do_pedido) AS ano, MONTH(Data_do_pedido) AS mes,  
SUM(Total_do_pedido) AS total_vendas FROM `tabela_pedidos` GROUP BY  
YEAR(Data_do_pedido), MONTH(Data_do_pedido) ORDER BY ano, mes;
```

Neste caso, estamos agrupando os dados primeiro pelo ano e, em seguida, pelo mês dentro desse ano, e em seguida, ordenando os resultados pelo ano e pelo mês.



LIKE

LIKE é uma ferramenta poderosa para realizar buscas flexíveis em colunas de texto no SQL. Com o uso adequado de curingas, você pode facilmente buscar padrões específicos e realizar filtragens avançadas em suas consultas.

Sintaxe:

```
SELECT coluna  
FROM nome_da_tabela  
WHERE nome_da_coluna LIKE  
'%valor_procurado%'
```



LIKE

Os caracteres curinga comuns usados com LIKE são:

- %: Substitui zero ou mais caracteres.
- _: Substitui exatamente um caractere.



LIKE

Correspondência no início:

Sintaxe:

```
SELECT nome_fornecedor  
FROM `tabela_fornecedores`  
WHERE nome LIKE 'T%';
```

Isso retornará todos os fornecedores cujos nomes começam com a letra "T".



LIKE

Correspondência no Final:

Sintaxe:

```
SELECT Nome_fornecedor  
FROM `tabela_fornecedores`  
WHERE Nome_fornecedor LIKE '%quality';
```

Isso retornará todos os clientes cujos nomes terminam com "quality"..



LIKE

Correspondência exata em uma posição específica:

Sintaxe:

```
SELECT Contato  
FROM `tabela_fornecedores`  
WHERE Contato LIKE '____@%';
```

Isso retornará todos os endereços de email que têm pelo menos quatro caracteres antes do símbolo "@".



LIKE

Correspondência geral:

Sintaxe:

```
SELECT Nome_Fornecedor  
FROM `tabela_fornecedores`  
WHERE Nome_Fornecedor LIKE '%tech%';
```

Isso retornará todos os fornecedores com a palavra "tech" em qualquer parte da descrição.



LIKE

A sensibilidade a maiúsculas e minúsculas do **LIKE** pode variar dependendo do banco de dados. Em alguns bancos de dados, como **MySQL**, **LIKE** é insensível a maiúsculas e minúsculas por padrão. No entanto, em outros, como **PostgreSQL**, **LIKE** é sensível a maiúsculas e minúsculas por padrão. Para buscas insensíveis a maiúsculas e minúsculas em **PostgreSQL**, você pode usar **ILIKE**.



AULA 9

Consultas Condicionais

Uma expressão condicional pode ser verdadeira (V) ou falsa (F). Quando for verdadeira, o elemento avaliado será exibido no resultado da consulta. Quando for falsa, será omitido.

Esse resultado depende do uso do sinal lógico, que pode ser o de igualdade (=), de diferença ($\neq$), maior ($>$), menor ($<$), maior ou igual ($\geq$), menor ou igual ($\leq$). Caso a caso, podemos verificar se uma expressão é verdadeira ou falsa, e analisar os dados resultantes.



Os filtros que criamos até então são bastante simples, portanto vamos incrementá-los colocando os operadores AND e OR no meio das expressões. Por exemplo:

```
SELECT * FROM tab WHERE X = A OR Y = B;
```

```
SELECT * FROM tab WHERE X = A AND Y = B;
```

Cada uma das expressões em uma condição pode ser verdadeira ou falsa e, a depender do operador lógico entre elas, a expressão como um todo (a condição completa) também poderá ser considerada verdadeira ou falsa. Vamos ver isso mais a fundo a seguir.

Operador OR (ou)

O operador **OR** determina que, se uma das expressões for verdadeira, a expressão completa será verdadeira. Somente no caso de as duas expressões serem falsas que a expressão completa também será falsa. Veja uma tabela que resume os possíveis resultados para $X = A \text{ OR } Y = B$:

Condição	Resultado
(V) OR (V)	Verdadeiro
(V) OR (F)	Verdadeiro
(F) OR (V)	Verdadeiro
(F) OR (F)	Falso

Operador Lógico AND

O operador **AND** estabelece que, somente se todas as expressões forem verdadeiras, a expressão completa também será verdadeira. Se uma delas for falsa, a expressão toda é falsa. Veja, a seguir, uma tabela que resume essa explicação:

Condição	Resultado
(V) OR (V)	Verdadeiro
(V) OR (F)	Falso
(F) OR (V)	Falso
(F) OR (F)	Falso

Outro operador importante é o NOT, que é aplicado para inverter o resultado de uma expressão. Observe o seguinte exemplo:

$\text{NOT } (X = A \text{ OR } Y = B)$

$\text{NOT } (X = A \text{ AND } Y = B)$

No primeiro caso, se a expressão interna $X = A \text{ OR } Y = B$ for verdadeira, o operador NOT torna o resultado falso. Por outro lado, se a expressão interna for falsa, colocando o NOT acabamos com uma expressão verdadeira. O mesmo serve para o segundo caso: se a expressão interna $X = A \text{ AND } Y = B$ for verdadeira, o resultado com o NOT será falso e vice-versa.

Veja a tabela que resume as possíveis conclusões para **NOT (X = A OR Y = B)**.
O resultado só será verdadeiro quando ambas expressões forem falsas:

Condição	Resultado
(V) OR (V)	Falso
(V) OR (F)	Falso
(F) OR (V)	Falso

E, para **NOT (X = A AND Y = B)**, o resultado só será falso quando ambas expressões forem verdadeiras:

Condição	Resultado
(V) OR (V)	Falso
(V) OR (F)	Verdadeiro
(F) OR (V)	Verdadeiro
(F) OR (F)	Verdadeiro



Exemplo: A condição é $X = A \text{ OR } Y = B$. Dessa vez, o MySQL percorrerá o campo "X" (comparando-o com A) e o campo "Y" (comparando-o com B), gravando os resultados parciais de cada expressão.

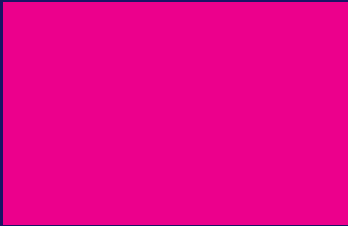
Como se trata de uma condição com o operador OR, sabemos que somente quando as duas expressões forem falsas que a expressão como um todo também será falsa:



Exercício 1

X	Y	Expr1		Expr2		Result
A	B	A = A	V	B = B	V	V
C	E	C = A	F	E = B	F	F
A	F	A = A	V	F = B	F	V
F	B	F = A	F	B = B	V	V

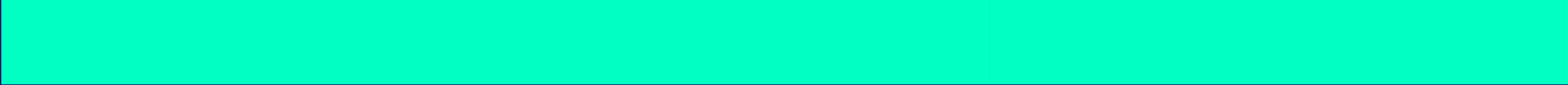
Ou seja, com base na nossa tabela, ao realizar uma consulta com a condição **X = A OR Y = B**, nosso retorno terá três registros (os verdadeiros).



Exemplo: A condição é $X = A \text{ AND } Y = B$. O processo será parecido com o anterior, porém dessa vez estamos trabalhando com o operador AND. Ou seja, somente quando as duas expressões forem verdadeiras que a expressão como um todo também será verdadeira:

X	Y	Expr1		Expr2		Result
A	B	A = A	V	B = B	V	V
C	E	C = A	F	E = B	F	F
A	F	A = A	V	F = B	F	F
F	B	F = A	F	B = B	V	F

O retorno dessa consulta terá apenas um registro (a primeira linha).



A expressão abaixo é verdadeira ou falsa?

$(\text{NOT } ((3 > 2) \text{ OR } (4 \geq 5)) \text{ AND } (5 > 4)) \text{ OR } (9 > 0)$





RESOLUÇÃO:

$(\text{NOT } ((3 > 2) \text{ OR } (4 \geq 5)) \text{ AND } (5 > 4)) \text{ OR } (9 > 0)$

$(\text{NOT } ((\text{Verdadeiro}) \text{ OR } (\text{Falso})) \text{ AND } (\text{Verdadeiro})) \text{ OR } (\text{Verdadeiro})$

$(\text{NOT } (\text{Verdadeiro}) \text{ AND } (\text{Verdadeiro})) \text{ OR } (\text{Verdadeiro})$

$(\text{Falso AND Verdadeiro}) \text{ OR } (\text{Verdadeiro})$

$(\text{Falso}) \text{ OR } (\text{Verdadeiro})$

Verdadeiro





JOIN

Um JOIN em SQL é uma forma de combinar dados de duas ou mais tabelas com base em uma coluna que elas compartilham. Essa coluna em comum é geralmente uma chave, como um ID.

O JOIN serve para conectar essas tabelas e permitir que você visualize dados relacionados em uma única consulta.





Tipos de JOIN:

1. **INNER JOIN:** Retorna apenas os registros que têm correspondência nas duas tabelas.
2. **LEFT JOIN:** Retorna todos os registros da tabela da esquerda, mesmo que não haja correspondência na tabela da direita. Se não houver correspondência, os dados da tabela da direita aparecem como nulos.
3. **RIGHT JOIN:** Similar ao LEFT JOIN, mas retorna todos os registros da tabela da direita, mesmo sem correspondência na esquerda.
4. **FULL OUTER JOIN:** Retorna todos os registros de ambas as tabelas, incluindo aqueles sem correspondência.



Tabela **Clientes** :

id_cliente	nome	cidade
1	Maria	São Paulo
2	João	Rio de Janeiro
3	Ana	Salvador

Tabela **Pedidos** :

id_pedido	id_cliente	produto
101	1	Laptop
102	1	Celular
103	2	Tablet

E se quisermos ver quais clientes fizeram pedidos e o que eles compraram?

```
SELECT Clientes.nome, Pedidos.produto  
FROM Clientes  
INNER JOIN Pedidos ON Clientes.id_cliente = Pedidos.id_cliente;
```



REVISÃO DO CONTEÚDO



- Ao longo deste curso, vocês adquiriram uma compreensão dos fundamentos de banco de dados e da linguagem SQL, que são essenciais para a manipulação e consulta de dados.
- SQL é uma ferramenta poderosa usada em quase todas as indústrias que lidam com dados, como finanças, saúde, comércio eletrônico e muito mais.
- Dominar comandos como SELECT, INSERT, UPDATE e DELETE oferece a habilidade de acessar e gerenciar grandes volumes de dados de forma eficiente e precisa.

Pontos Chave:

- Compreensão dos principais comandos SQL: saber como consultar, inserir, atualizar e deletar dados.
- Uso de chaves primárias e estrangeiras: garantindo a integridade e os relacionamentos entre tabelas.
- Filtros e funções agregadas: aprender a refinar consultas para obter resultados específicos.





Com a prática, essas habilidades os ajudarão a navegar por grandes bancos de dados e a tomar decisões informadas baseadas em dados reais. Continuem praticando e experimentando para fortalecer seu conhecimento e acompanhar as exigências do mercado de trabalho.

